Machine Learning Operations

Lecture 1: Introduction and ML Fundamentals

Daniel Dahlmeier, Christopher Teo

December 13, 2025

Agenda

- Course overview
- Introduction
- When to use and when not to use ML
- Recap of fundamental ML concepts

Course Overview

- About this course
  - Machine Learning Operations (MLOps)
  - Instructor: Daniel Dahlmeier & Christopher Teo
- Learning objectives
  - Learn how to build ML products
  - Identify viable ML use cases
  - Identify legal, data protection and ethical risks
  - Develop, test, deploy and monitor ML applications
  - Manage machine learning projects and teams
  - *Not:* learn fundamentals of machine learning

Credit: Full stack deep learning (FSDL), deeplearning.ai, Stanford's CS329 Machine learning Systems Design course and many more

MLOps

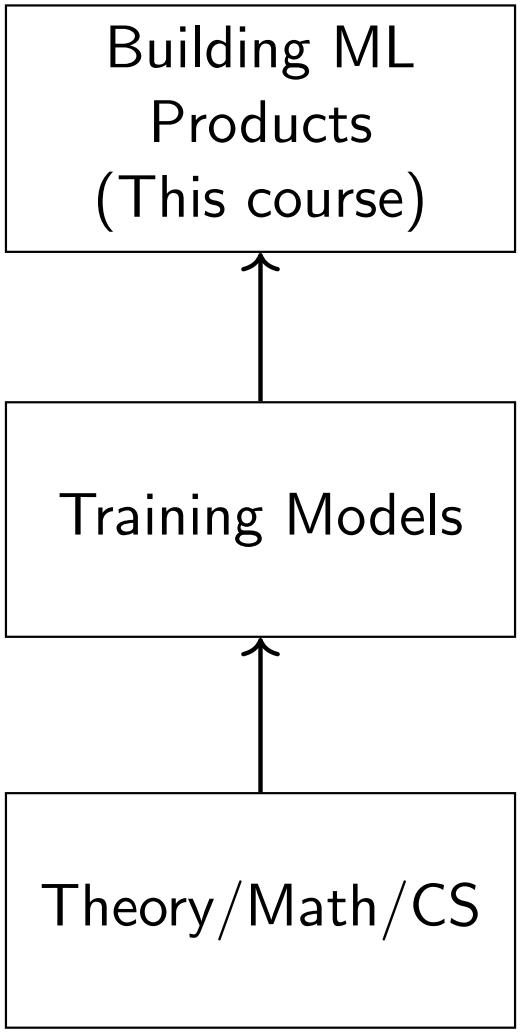
$$\text{ML} + \text{DevOps} = \text{MLOps}$$

(Now people are already talking about LLMOps)

MLOps

- "DevOps for ML"
- Set of practices to standardize and streamline ML development
- Develop and deploy ML into production quickly, safely, and securely
- Create fast feedback loops
- Release ML features in a controlled, predictable, and reversible manner
- Builds on DevOps and agile software development practices

The focus of this course



Quiz time: meet the class

- What is your background in ML?
- What are your expectations for this course?
- Do you already have experience building an ML product?

About me

Christopher Teo

- Academia
  - Ph.D ISTD; Bs.c EPD from SUTD
  - Research on Computer Vision — Fairness in generative models.
- Industry
  - Worked as a Product Engineer at AMD
  - Currently a Data Scientist at SAP

About me

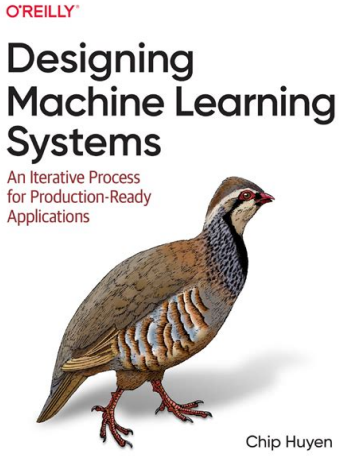
Daniel Dahlmeier

- Academia
  - Master CS KIT (Karlsruhe), Ph.D NUS (Singapore), MBA MBS (Mannheim)
  - Research on grammatical error correction, MT, and sentiment analysis
  - Adjunct assistant professor Singapore University of Technology and Design (SUTD)
- Industry
  - Worked as researcher, data scientist, and manager at SAP
  - Currently Chief Data Scientist and team lead at SAP Artificial Intelligence in Singapore

Course Logistics

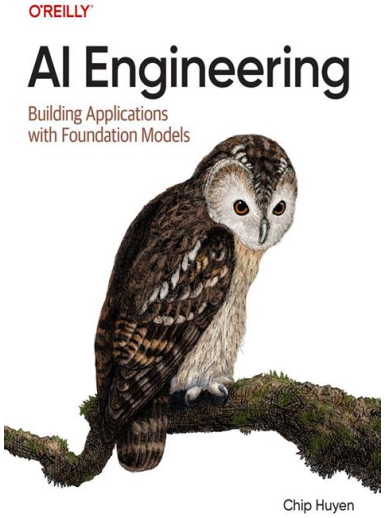
- Schedule and Assignments
  - Weekly lectures and hands-on sessions
  - 3 sessions per week: 2 lectures and 1 hands-on sessions for a total of 5 content hours per week.
  - Additional 7 hours per week on self-study work for a total of 12 hours effort per week.
- Faculty Contact Information
  - Instructor: Daniel Dahlmeier (daniel_dahlmeier@sutd.edu.sg) Christopher Teo (christopher_teo@alumni.sutd.edu.sg)
  - Teaching assistant: TBD
  - Office hours: email to make an appointment
- Tools and platforms
  - Email: announcements and general information
  - eDimensions: slides and course material
  - ed: discussion forum. Please sign up here
  - AWS and SUTD education cluster: hands-on sessions and assignments

Course text books



Designing Machine Learning Systems. Chip Huyen. 2022.

- Build applications with traditional ML models
- Feature engineering, model training, etc.



AI Engineering. Chip Huyen. 2025.

- Build application with foundation models
- Prompt engineering, context construction, finetuning

Both books are available in the SUTD library.

Assessment

- Individual Assignments 20%, 2 weeks to submit, individual performance
  - One in week 3: 10%
  - One in week 5: 10%
- Project / Group Assignments 20%, 2 weeks to submit, team performance, groups of 3-5 students
  - One in week 9: 10%
  - One in week 11: 10%
- Class Participation 10%, individual performance
- Exams 50%
  - Mid-term exam in week 8: 20%
  - Final exam in week 14: 30%
- Late Submission penalty
  - Between 0 and 6 hours late: 25%
  - 6 to 12 hours late: 50%
  - 12 to 18 hours late: 75%
  - Above 18 hours late: 100%

Project / Group Assignments

- Build a chatbot that can talk to prospective students and answer questions about SUTD
- Improve applicant's user experience on the SUTD website
- Perform user research, collect and create data sets, implement LLM-based experiments and prototype using RAG and LLM finetuning

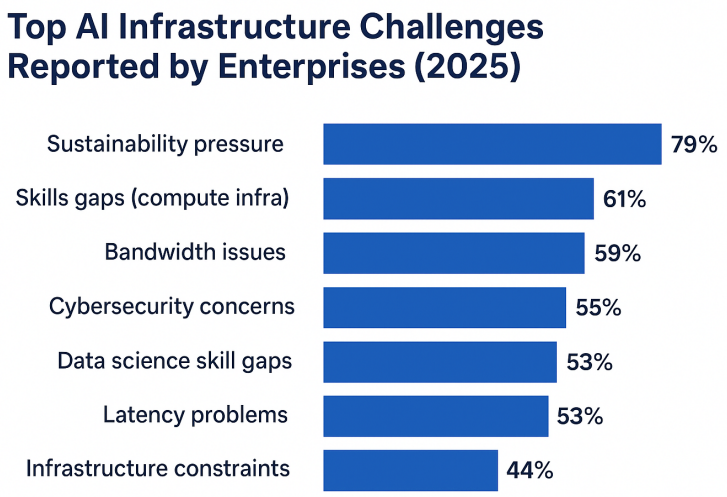
Introduction

Machine learning in 2026

- Breakthroughs in several AI/ML tasks in last 2 decades
- AI rivaling or exceeding human performance in many tasks in computer vision, speech recognition, games, task automation
- Explosion of generative AI models and applications
- AI becoming mainstream thanks to ChatGPT
- Domination of foundation models and transformer architecture
- Increasing public debate about the risks of AI and need for regulation
- Mainstream adoption of Agentic AI systems

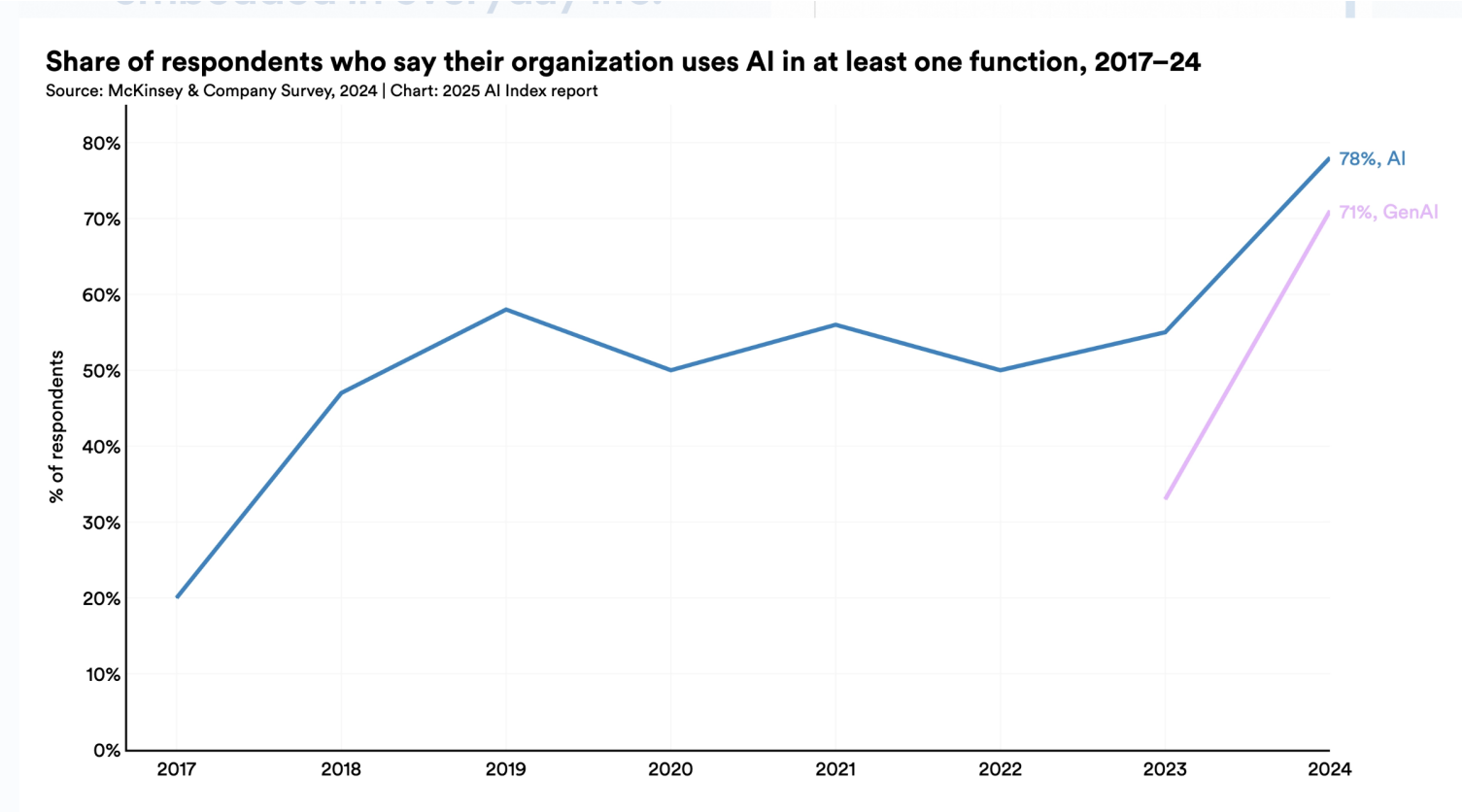
Companies believe AI is essential and start adopting, but challenges remain

- BCG Global Survey *(2024): 74% of companies struggle to achieve and scale value from AI.
- EY Work Reimagine Survey (2025): 88% of employees use AI at work, but mostly in basic ways
- Gartner (2025): 59% of finance leaders reported active AI use in finance operations
- IBM (2025): 45% of companies express concerns with accuracy and bias.



Data adopted from flexential

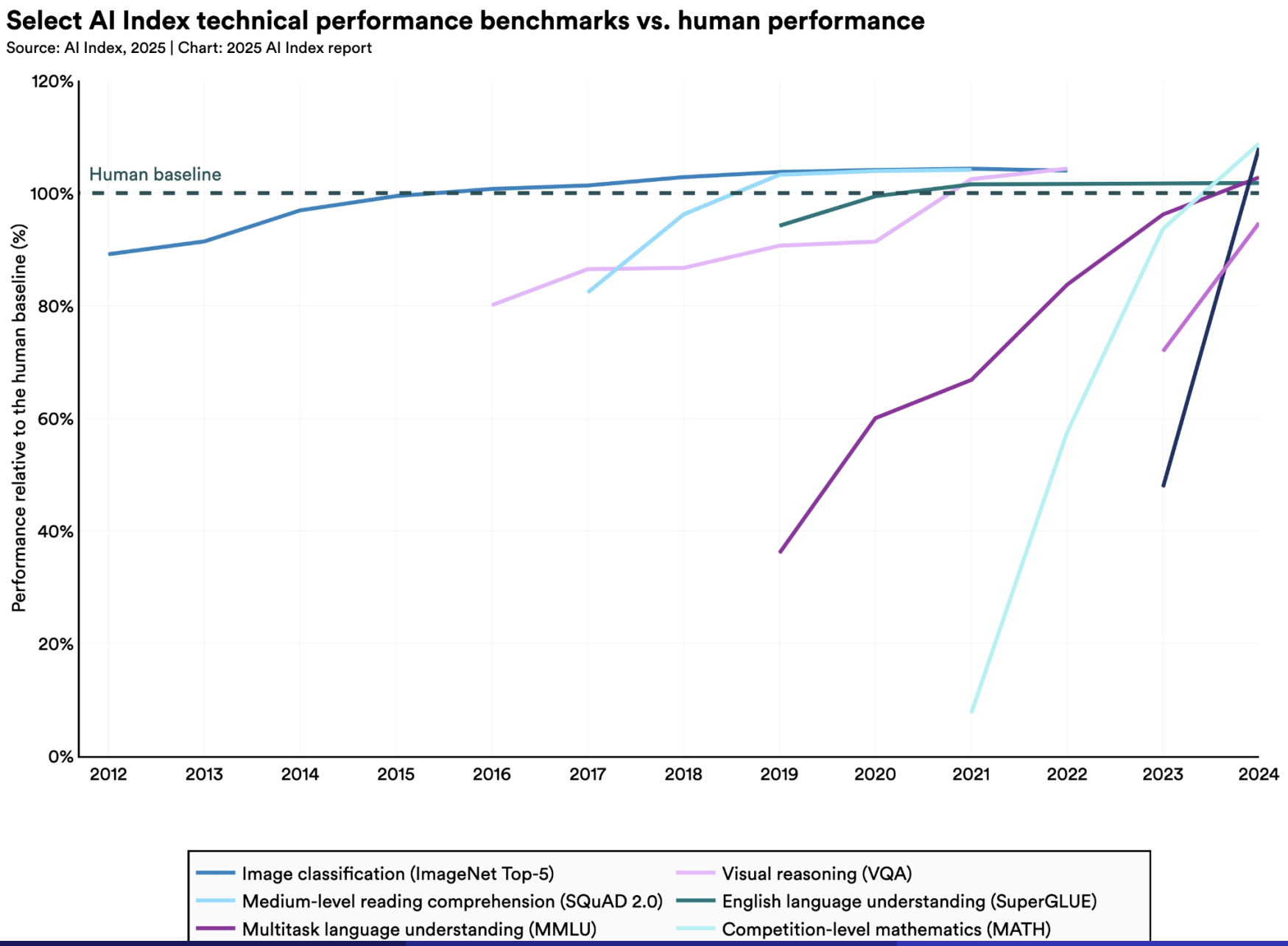
AI adoption



A 2023 McKinsey report reveals that 55% of organizations now use AI (including generative AI) in at least one business unit or function, up from 50% in 2022 and 20% in 2017.

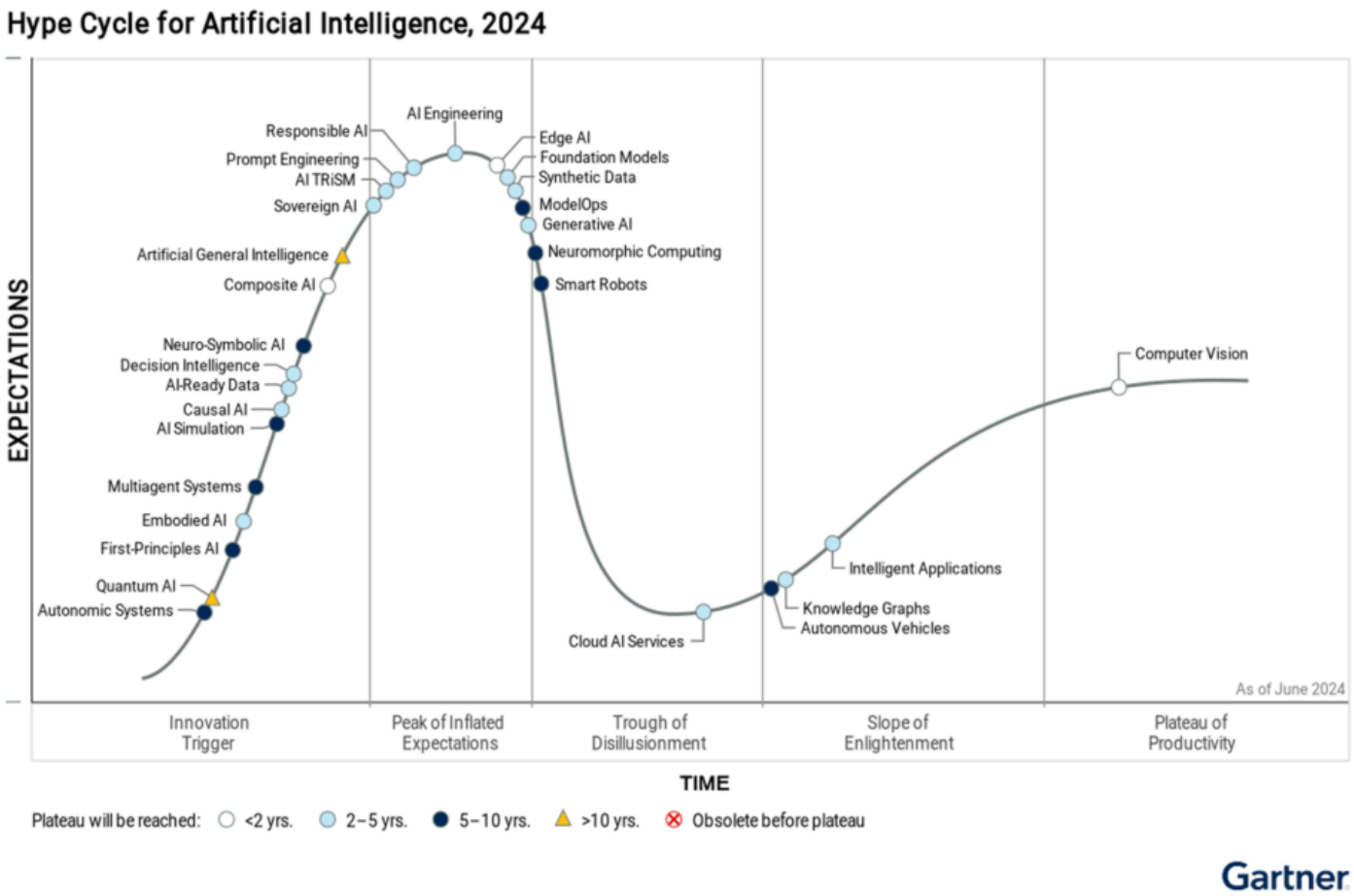
Stanford HAI AI Index Report 2024

Progress in AI has been accelerating

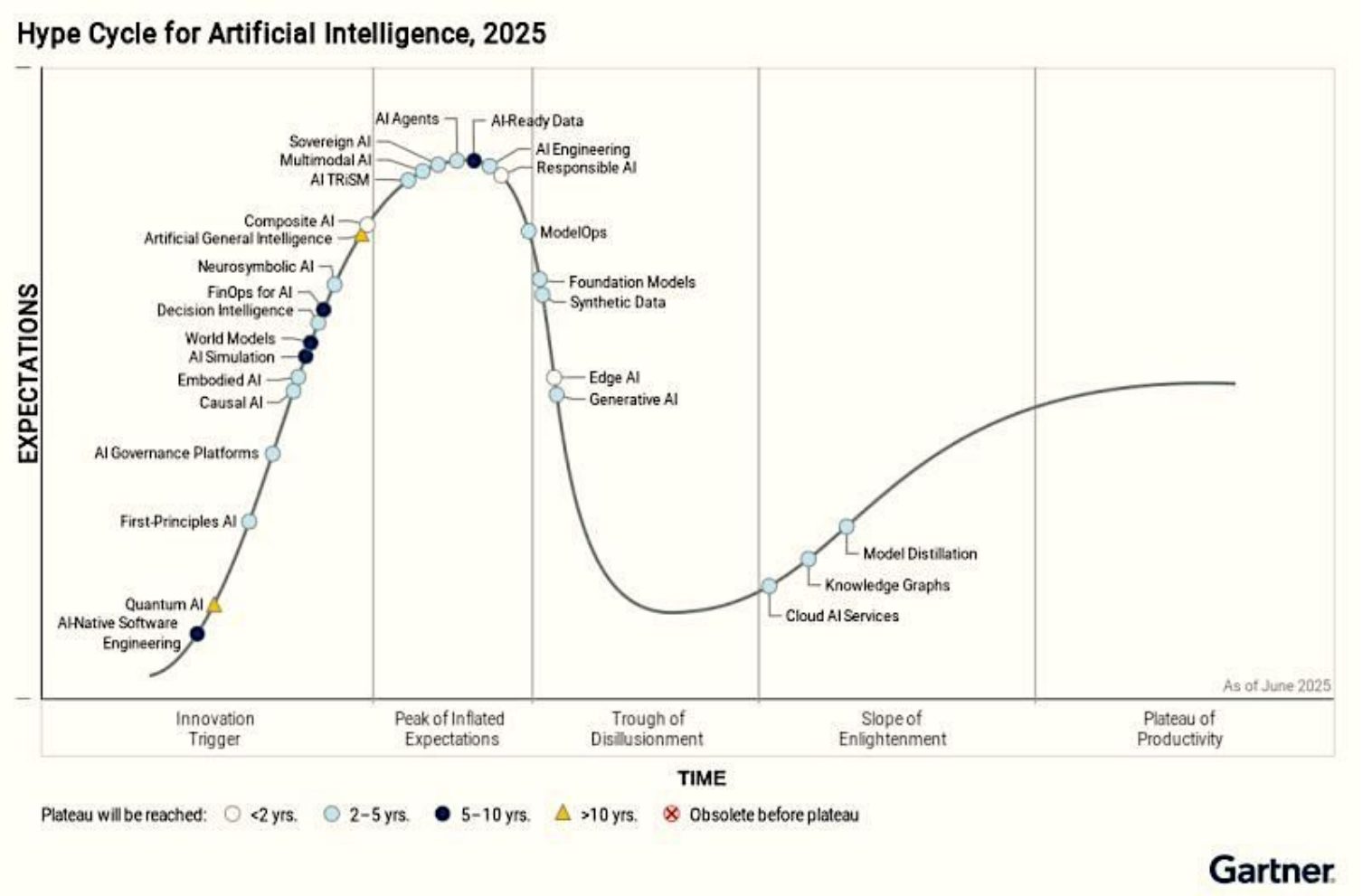


Gartner artificial intelligence hype cycle 2024

Figure 1: Hype Cycle for Artificial Intelligence, 2024



Gartner artificial intelligence hype cycle 2025

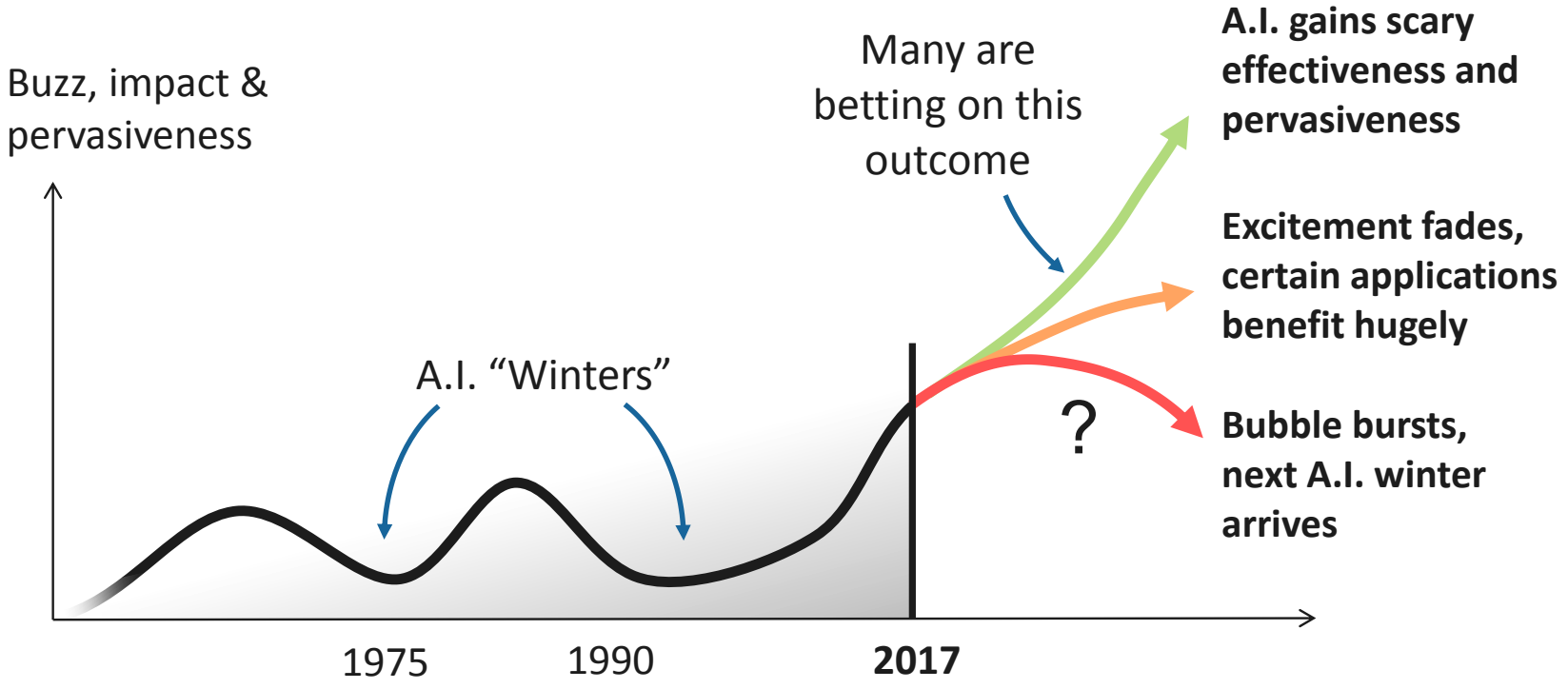


AI hype and AI winters

Cambridge Consultants

Most deployed AI is dumb or impotent

AI is enjoying significant hype and investment



Quiz

- Are we heading for
  - AI gains scary effectiveness and pervasiveness
  - Excitement fades, certain applications benefit hugely
  - Bubble bursts, next A.I. winter arrives

When to use and when not to use ML

Lots of ML projects fail(?)

- Commonly quoted statistic: 87% of ML projects fail but there seems to be no real evidence for this number [1]
- What does "failure" mean? just an idea which was not further developed? a model with poor accuracy? a technical prototype without a clear business model? a product with mediocre commercial success?

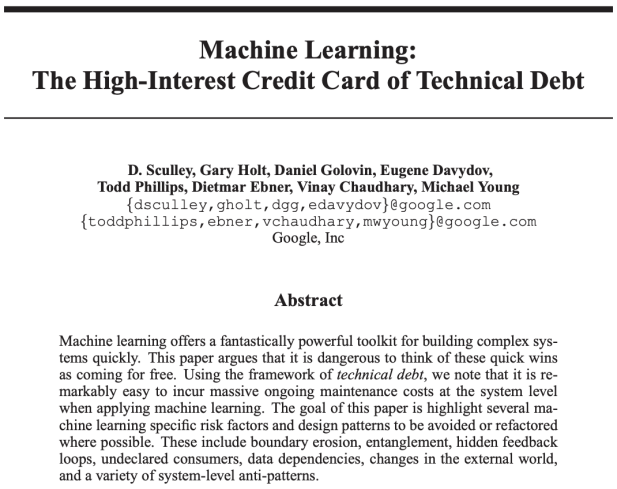
[1] https://mtszkw.medium.com/why-do-87-of-data-science-projects-fail-and-are-we-sure-that-it-is-true-fe8b5ba1404c

Why ML projects fail

- ML is still very much a research topic and less mature than other software components
- Reasons for ML project failure:
  - Project is poorly scoped - unclear what success even means (no DoD)
  - Technically not feasible with current technology
  - Works, but does not solve a big enough problem to be worth the complexity
  - Mismatch between business expectations and what is technically feasible

ML introduces a lot of complexity

- **Erodes boundaries between systems** e.g., data pipelines, model service (API), host service (MCP)
- **Relies on expensive data assets**
- **Commonly plagued by system anti-patterns** e.g., vesion control, testing, monitoring
- **Subject to the instability of the external world** e.g., domain shift



Machine Learning:
The High-Interest Credit Card of Technical Debt

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov,
Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young
{dsculley,gholt,dgg,edavydov}@google.com
{toddphillips,ebner,vchaudhary,mwyoung}@google.com
Google, Inc

Abstract

Machine learning offers a fantastically powerful toolkit for building complex systems quickly. This paper argues that it is dangerous to think of these quick wins as coming for free. Using the framework of *technical debt*, we note that it is remarkably easy to incur massive ongoing maintenance costs at the system level when applying machine learning. The goal of this paper is highlight several machine learning specific risk factors and design patterns to be avoided or refactored where possible. These include boundary erosion, entanglement, hidden feedback loops, undeclared consumers, data dependencies, changes in the external world, and a variety of system-level anti-patterns.

Figure: (Sculley et al. 2014)

Before starting an ML project, ask yourself:

- Are we ready to use ML?
- Do we really need ML to solve this problem?
- Do we have the data we need to build ML?
- Is it ethical and legal to use the data and ML?

Before starting an ML project, ask yourself:

- Are we ready to use ML?
  - Do we have a product?
  - Do we have the right people?
- Do we really need ML to solve this problem?
- Do we have the data we need to build ML?
- Is it ethical and legal to use the data and ML?

Before starting an ML project, ask yourself:

- Are we ready to use ML?
- Do we really need ML to solve this problem?
  - Do we need to solve the problem?
  - Have we tried using rules or simple stats?
- Do we have the data we need to build ML?
- Is it ethical and legal to use the data and ML?

Before starting an ML project, ask yourself:

- Are we ready to use ML?
- Do we really need ML to solve this problem?
- Do we have the data we need to build ML?
  - Do we have relevant training data?
  - Do we have labels?
- Is it ethical and legal to use the data and ML?

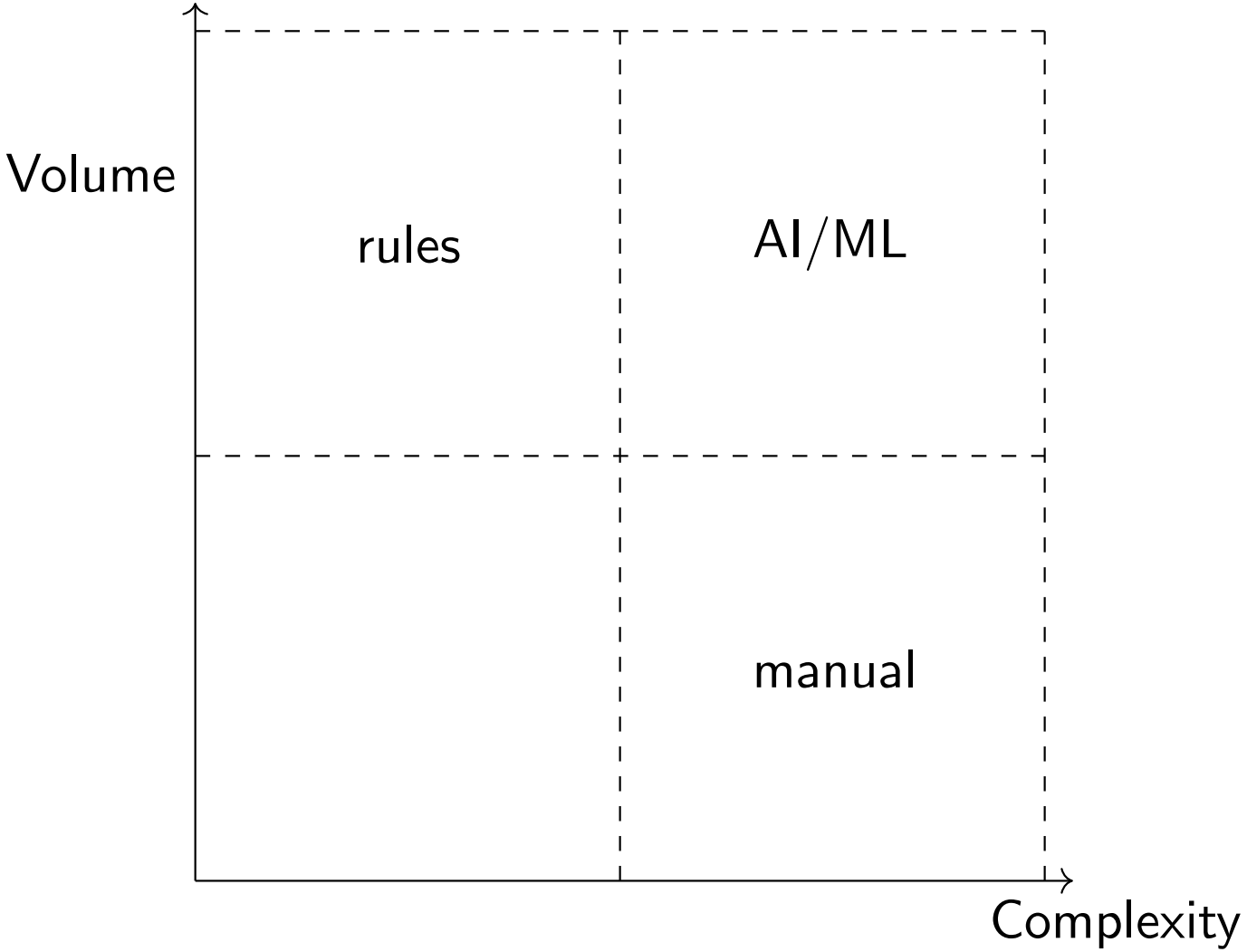
Before starting an ML project, ask yourself:

- Are we ready to use ML?
- Do we really need ML to solve this problem?
- Do we have the data we need to build ML?
- Is it ethical and legal to use the data and ML?
  - Should this application be built at all?
  - Is there potential harm from using ML?
  - Do we have a legal basis for using the data?
  - Do we respect the privacy and basic rights of our users?

How to pick problems to solve with ML: high impact, low cost

- High impact problems are likely to be those that address
  - Friction in your product
  - Complex parts of your pipeline
  - Places where cheap prediction is valuable
  - What other people in your industry are doing
- Low-cost projects are those with data available, and where bad predictions aren't too harmful

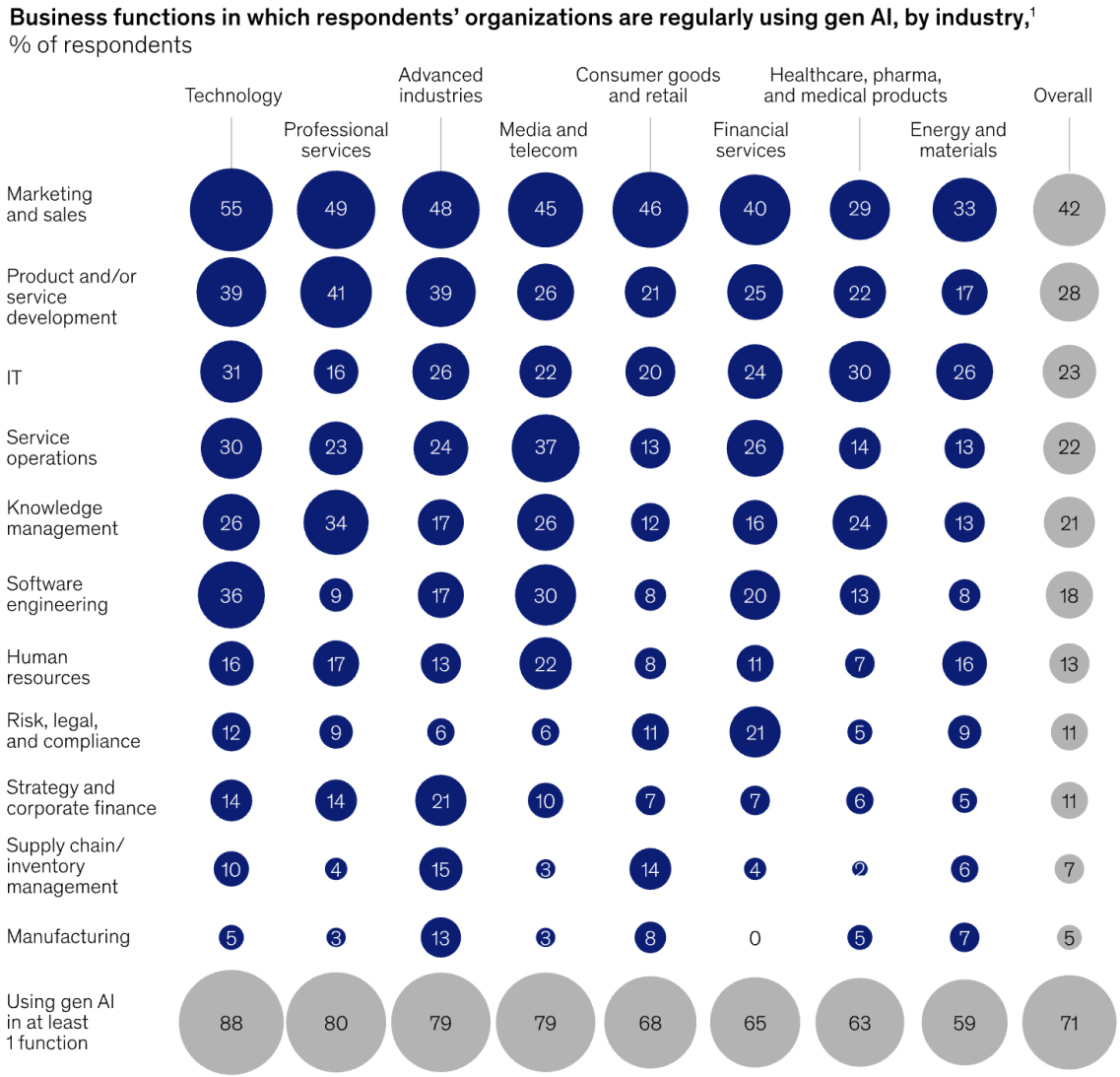
Task complexity vs. volume



How to pick problems to solve with ML

- Where can you take advantage of cheap prediction?
- Where is there friction in your product?
- Where can you automate complicated manual processes?
- Where do you have high volume decisions which are too complicated for rules?
- What are other people doing?

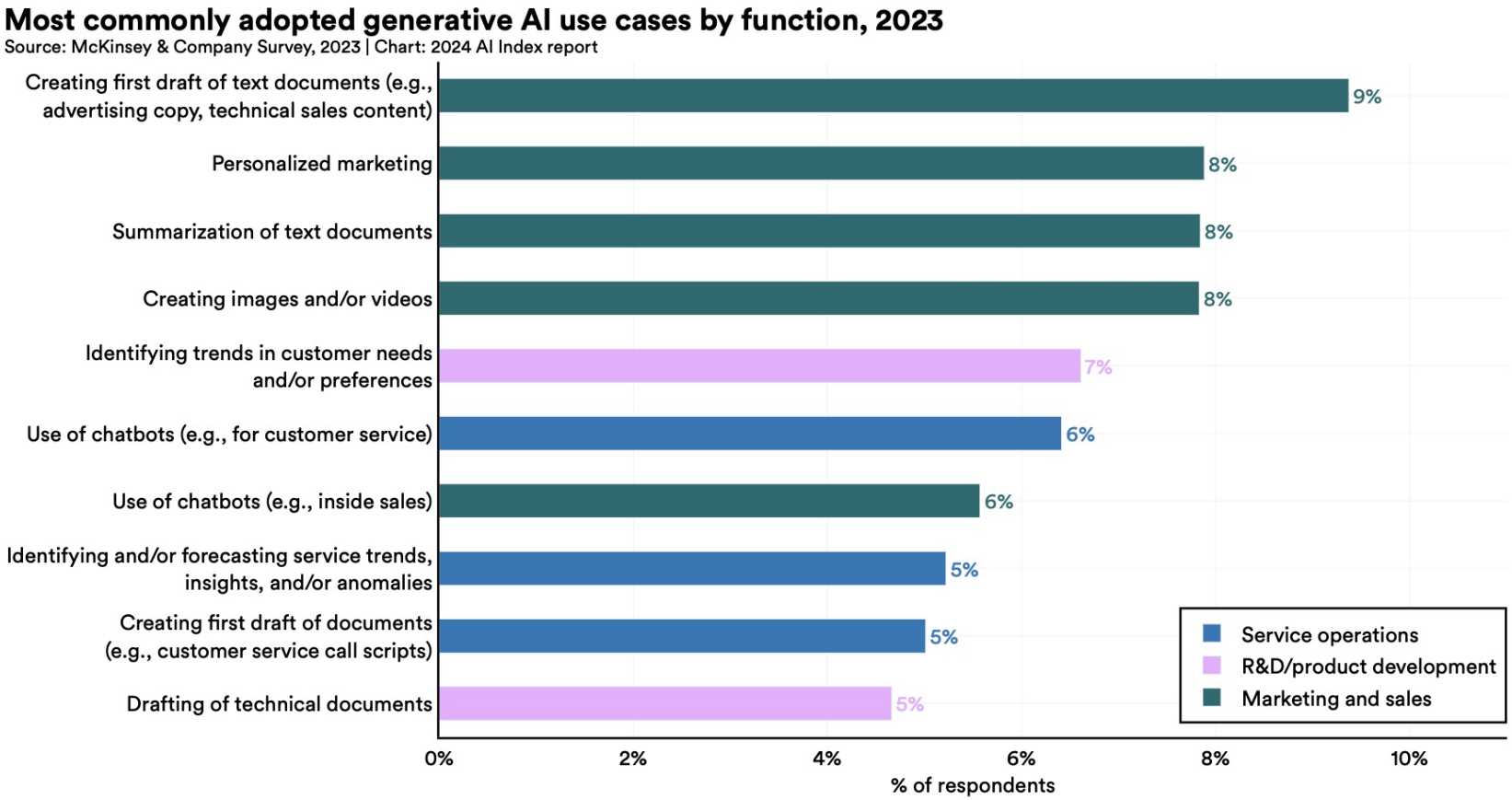
What are other people doing?



[1]For technology, n = 199; for business, legal, and professional services, n = 179; for media and telecom, n = 77; for advanced industries (includes advanced electronics, aerospace and defense, automotive and assembly, and semiconductors), n = 97; for financial services, n = 193; for consumer goods and retail, n = 111; for healthcare, pharma, and medical products, n = 113; and for energy and materials, n = 142.
Source: McKinsey Global Survey on the state of AI, 1,491 participants at all levels of the organization, July 16–31, 2024

McKinsey & Company

What are other people doing?



Stanford HAI AI Index Report 2024

Case studies

- Using Machine Learning to Predict Value of Homes On Airbnb (Robert Chang, Airbnb Engineering & Data Science, 2017)
- Using Machine Learning to Improve Streaming Quality at Netflix (Chaitanya Ekanadham, Netflix Technology Blog, 2018)
- 150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com (Bernardi et al., KDD, 2019)
- How we grew from 0 to 4 million women on our fashion app, with a vertical machine learning approach (Gabriel Aldamiz, HackerNoon, 2018)
- Machine Learning-Powered Search Ranking of Airbnb Experiences (Mihajlo Grbovic, Airbnb Engineering & Data Science, 2019)

Designing Machine Learning Systems (https://github.com/chiphuyen/dmls-book/blob/main/resources.md)

Case studies (contd.)

- From shallow to deep learning in fraud (Hao Yi Ong, Lyft Engineering, 2018)
- Space, Time and Groceries (Jeremy Stanley, Tech at Instacart, 2017)
- Creating a Modern OCR Pipeline Using Computer Vision and Deep Learning (Brad Neuberg, Dropbox Engineering, 2017)
- Scaling Machine Learning at Uber with Michelangelo (Jeremy Hermann and Mike Del Balso, Uber Engineering, 2019)
- Spotify's Discover Weekly: How machine learning finds your new music (Sophia Ciocca, 2017)

Designing Machine Learning Systems (https://github.com/chiphuyen/dmls-book/blob/main/resources.md)

Assessing feasibility of ML projects

Cost drivers

- **Problem difficulty**
- Accuracy requirements
- Data availability

Main considerations

- Is the problem well-defined?
- Good published work on similar problems? (newer problems mean more risk and more technical effort)
- Compute requirements?
- Can a human do it? (Sanity check)

Assessing feasibility of ML projects

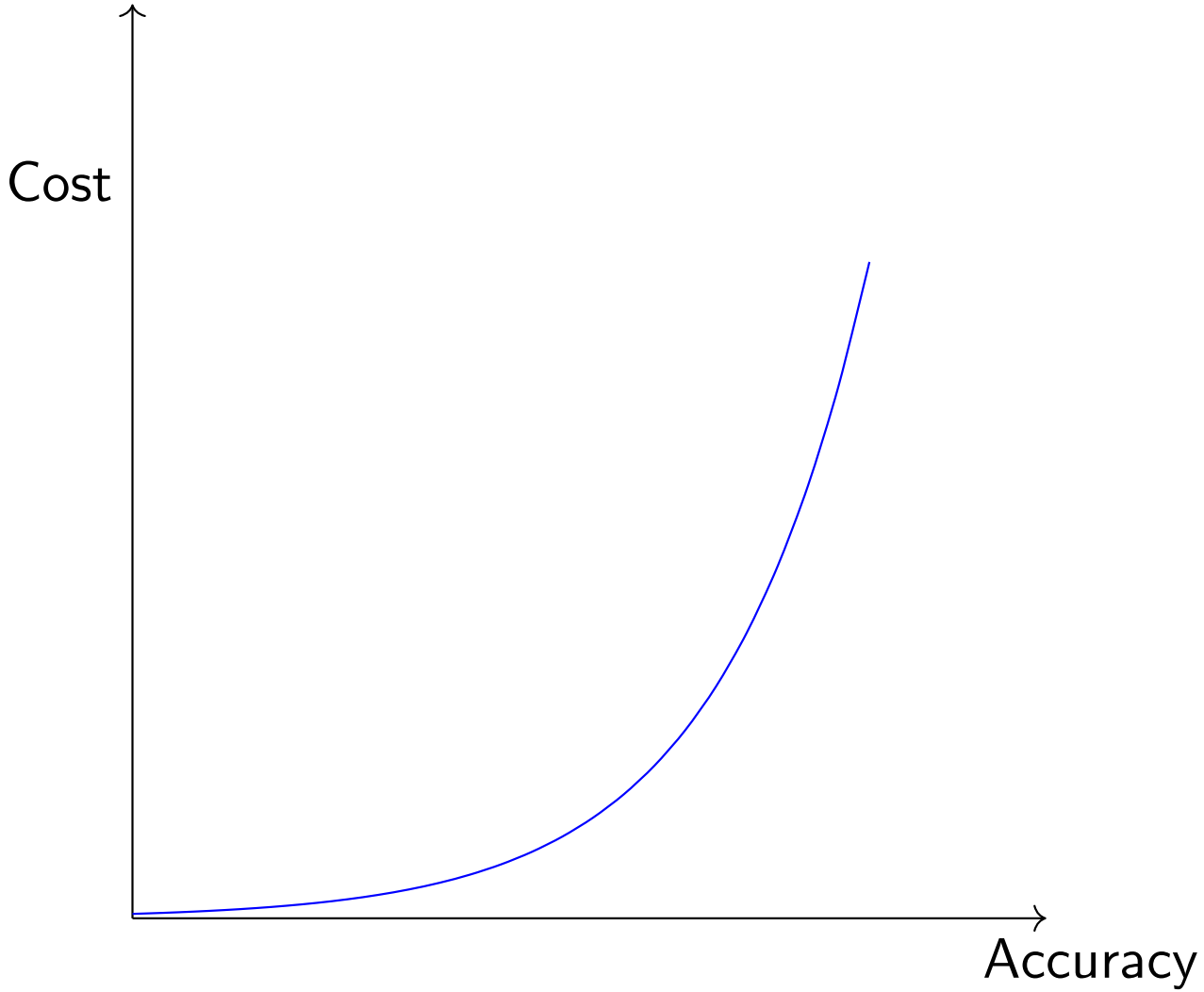
Cost drivers

- Problem difficulty
- **Accuracy requirements**
- Data availability

Main considerations

- How costly are wrong predictions?
- How frequently does the system need to be right to be useful
- Ethical implications?

Why are accuracy requirements so important?



ML project costs tend to scale super-linearly in the accuracy requirements

Why are accuracy requirements so important?

xkcd

Why are accuracy requirements so important?

The comic is from 2014. Now this task is easily solved by ChatGPT and multimodal AI models!

Assessing feasibility of ML projects

Cost drivers

- Problem difficulty
- Accuracy requirements
- **Data availability**

Main considerations

- How hard is it to acquire data?
- How expensive is data labeling?
- How much data will be needed?
- How stable is the data?
- Legal basis for using data?
- Copyright and licenses?
- Data protection and security requirements?

What type of problems are hard?

Easy

- Pattern Recognition (e.g., image classification)
- High Level Reasoning (e.g., chess)
- Complex Calculation and Prediction (e.g., Sales foreasting)
- Semantic Search (e.g., web search)

Challenging

- Reasoning and Explainability (e.g., medical diagnosis)
- Out-of-distribution Generalization (e.g., self-driving)
- Memory consistency (e.g., dialog systems)

Predicting the future of ML is hard

Tesla self-driving car demo from 2016

- Predictions on the arrival of autonomous vehicles (AVs) for over a decade
- Today: some autonomous taxi services but mass market AVs might still be decades away

How to run a ML feasibility assessment

1. Are you sure you need ML at all?
2. Put in the work up-front to define success criteria with all stakeholders
3. Consider the ethics of using ML
4. Do a literature review
5. Try to rapidly build a labeled benchmark dataset
6. Build a *minimum* viable model (e.g., manual rules)

Machine learning product archetypes

Software 2.0: Taking something software does today and doing it better with ML

Examples

- Improve code completion in an IDE
- Build a customized recommendation system
- Build a better video game AI

Key Questions

- Do your models truly improve performance?
- Does performance improvement generate business value?
- Do performance improvements lead to a data flywheel?

Machine learning product archetypes

Human in the loop: Helping humans do their jobs better by complementing them with ML-based tools

Examples

- Turn sketches into slides
- Email auto-completion
- Help a radiologist do their job faster

Key Questions

- How good does the system need to be to be useful?
- How can you collect enough data to make it that good?

Machine learning product archetypes

Autonomous systems: Taking something humans do today and automating it with ML

Examples

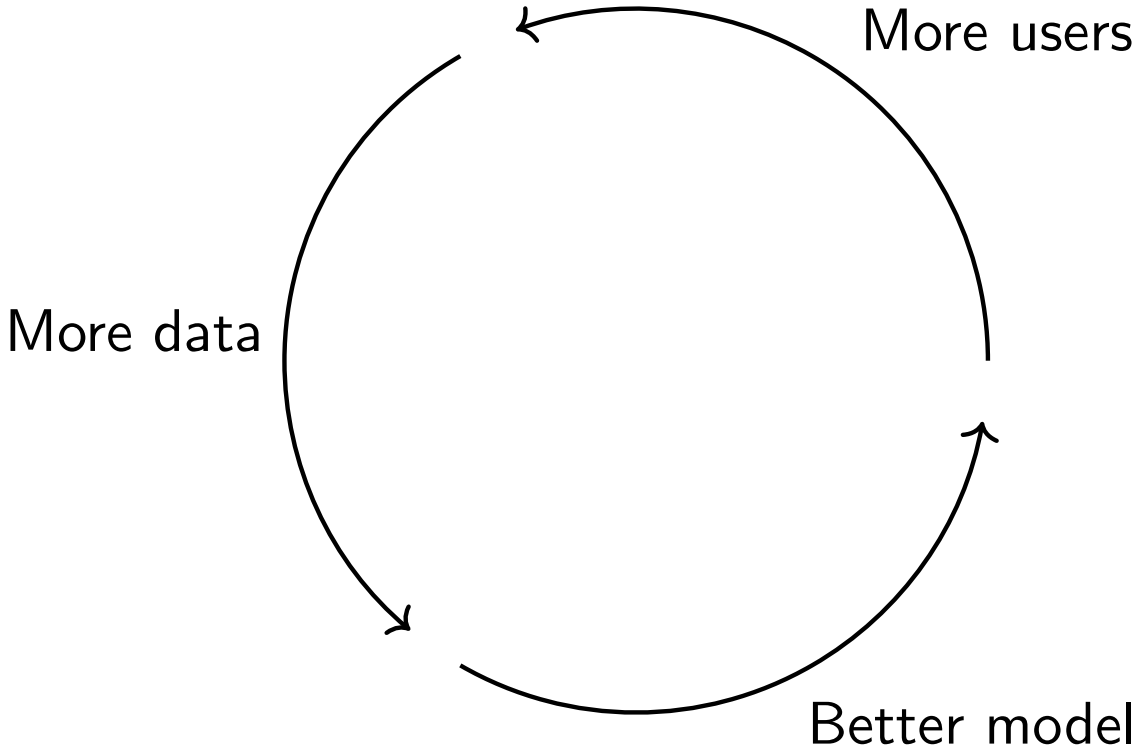
- Full self-driving
- Automated customer support
- Automated website design

Key Questions

- What is an acceptable failure rate for the system?
- How can you guarantee that it won't exceed that failure rate?
- How inexpensively can you label data from the system?

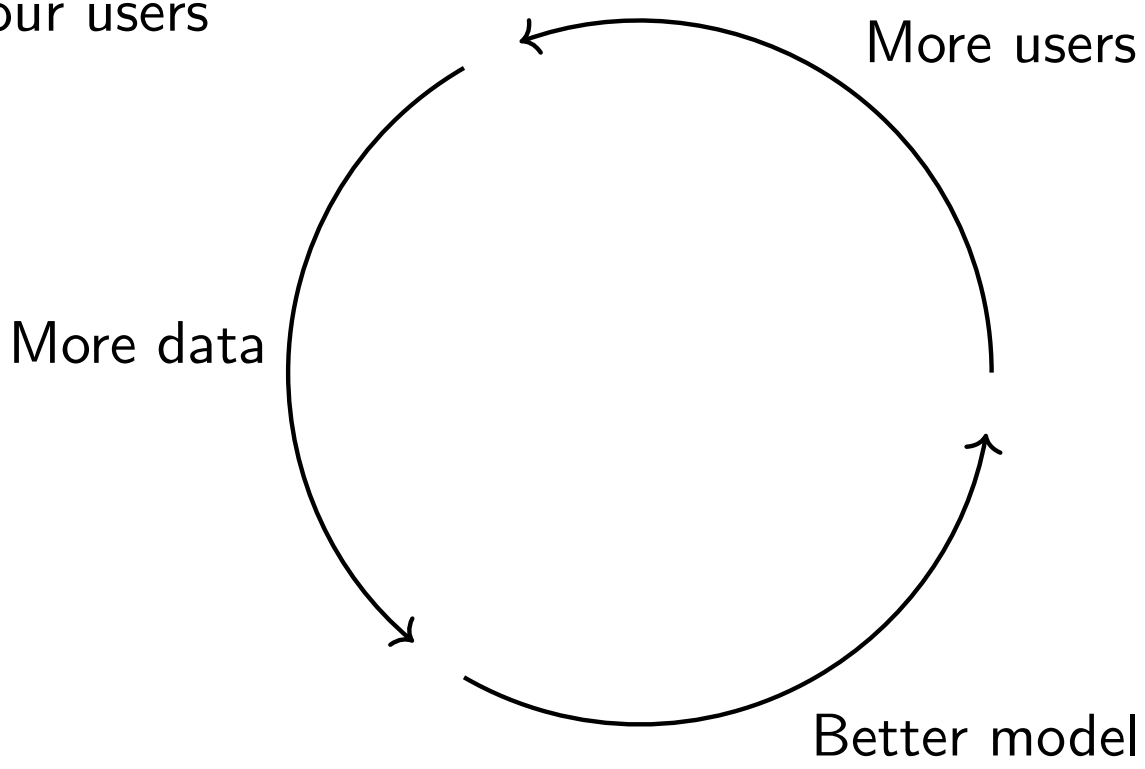
Data flywheels

Data flywheels – self-reinforcing, positive feedback loops where better models lead to more users, which leads to more data, which leads to better models.

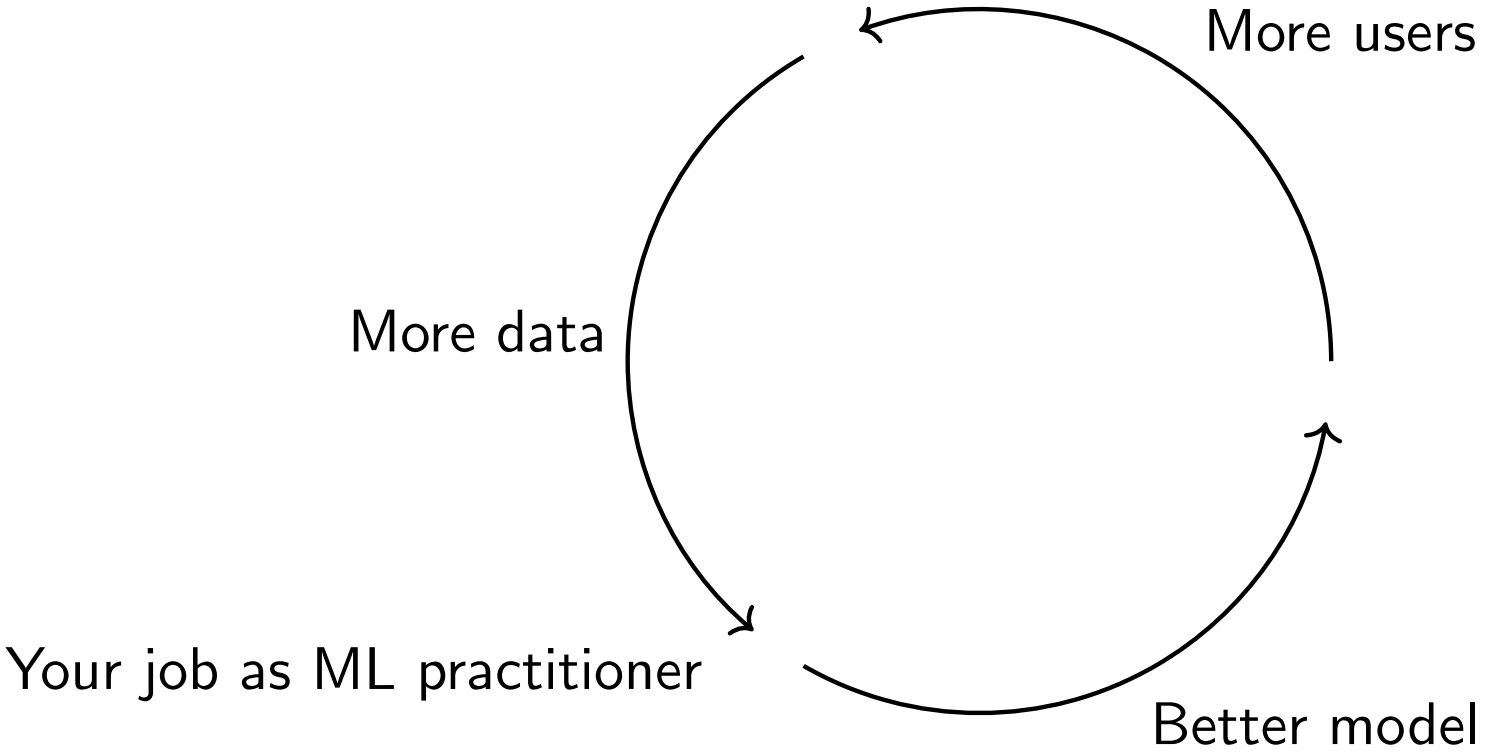


Data flywheels

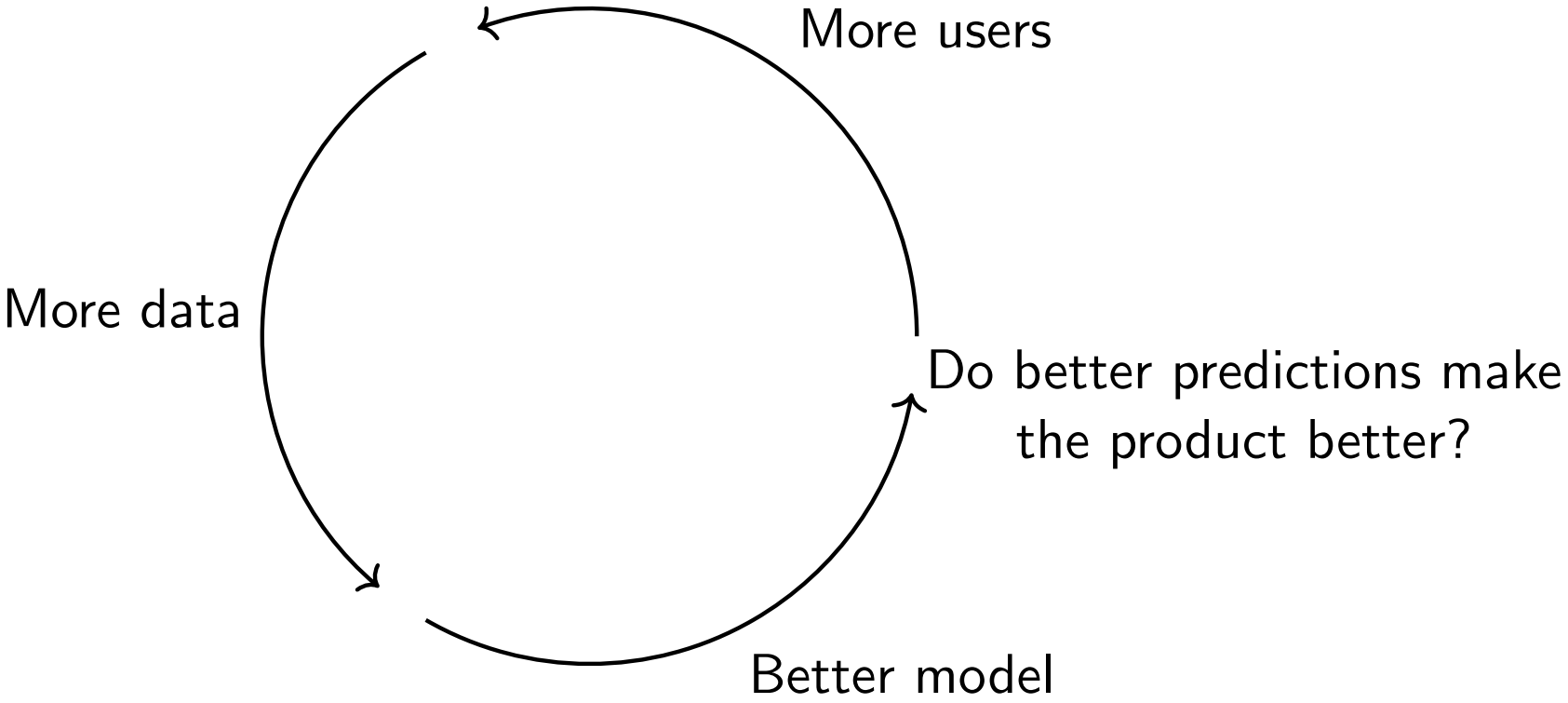
Do you have a data loop?
Automatically collect data
(and ideally labels)
from your users



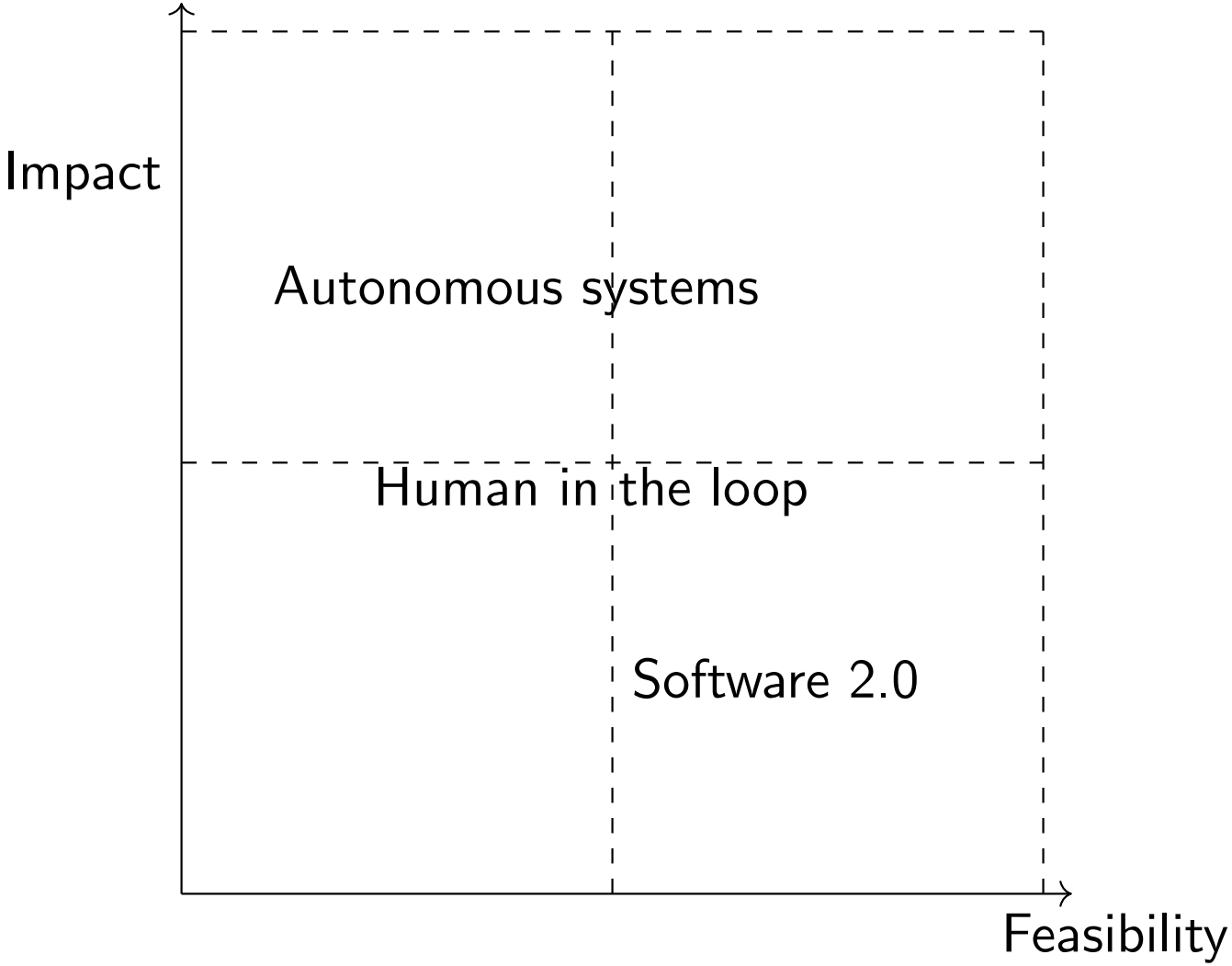
Data flywheels



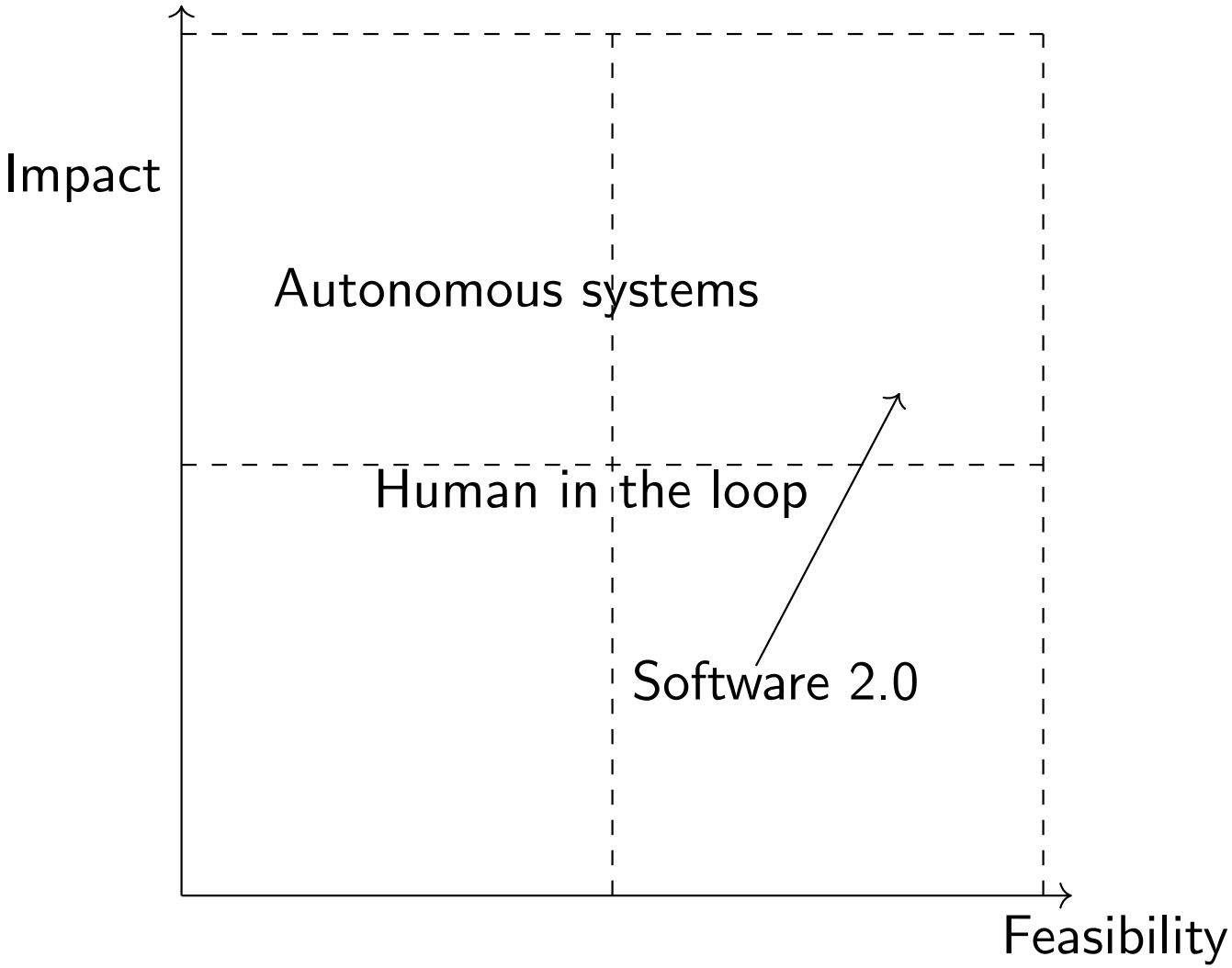
Data flywheels



Machine learning product archetypes

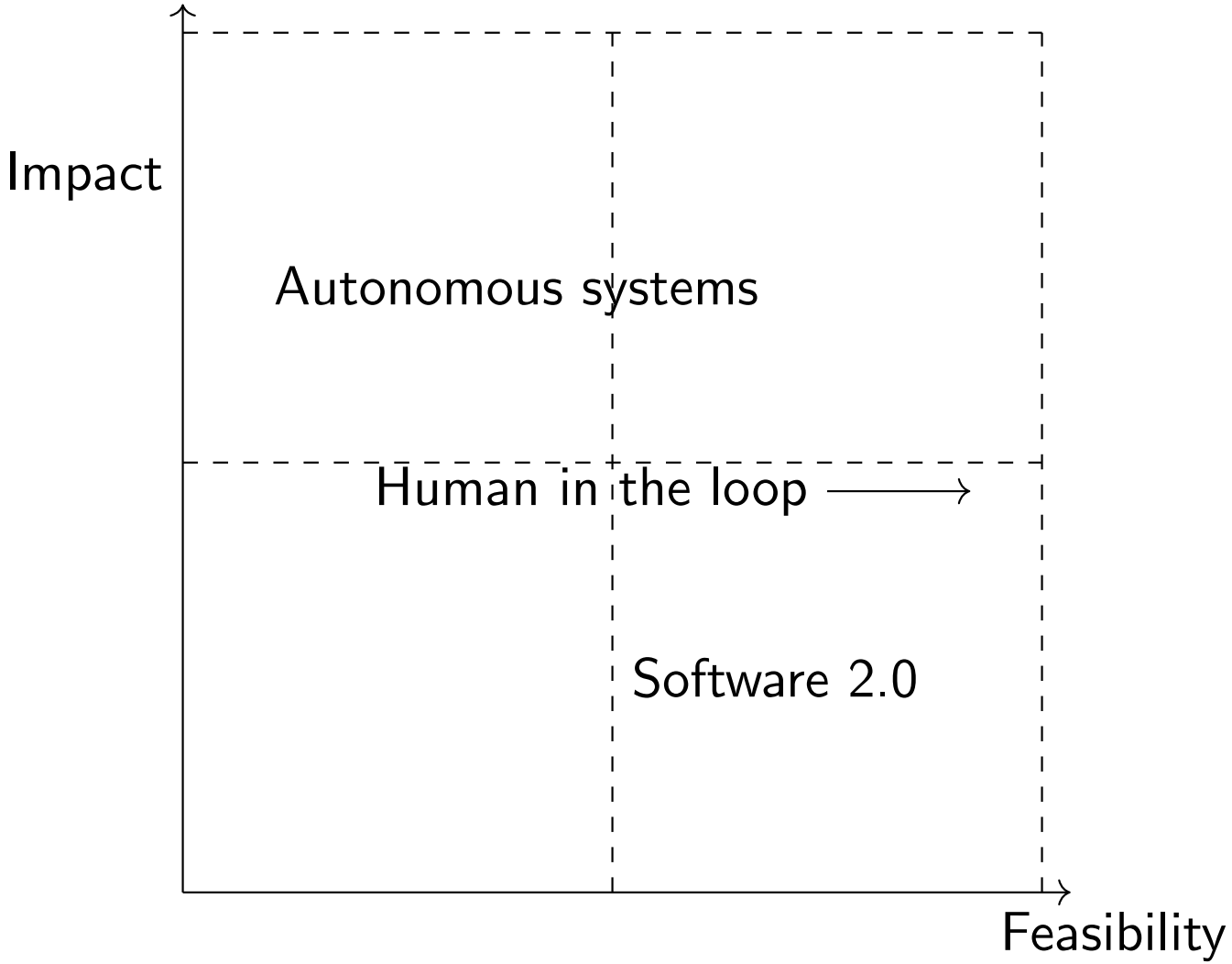


Machine learning product archetypes



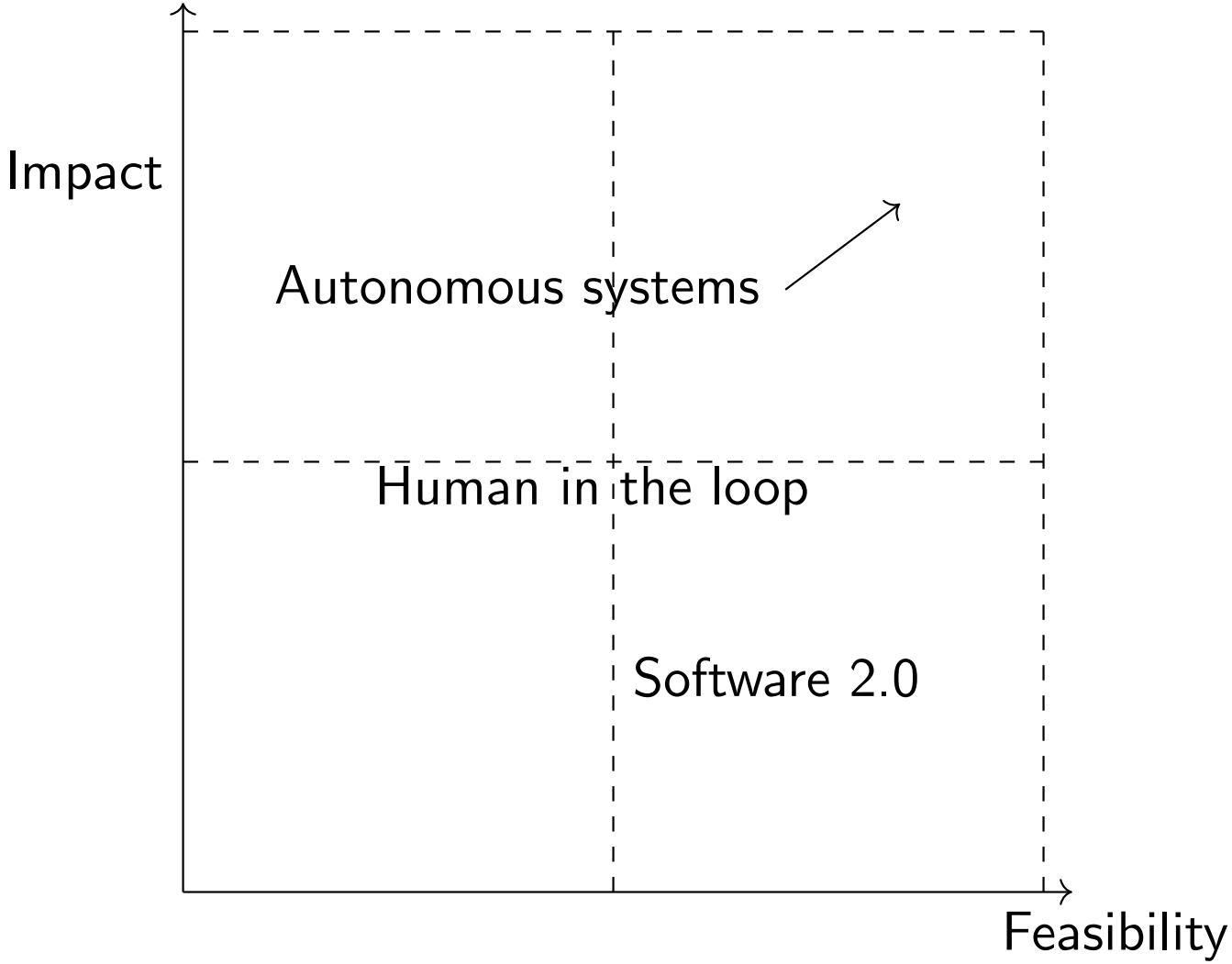
Implement a data loop that allows you to improve on this task and future ones

Machine learning product archetypes



Good product design. Release a 'good enough' version

Machine learning product archetypes



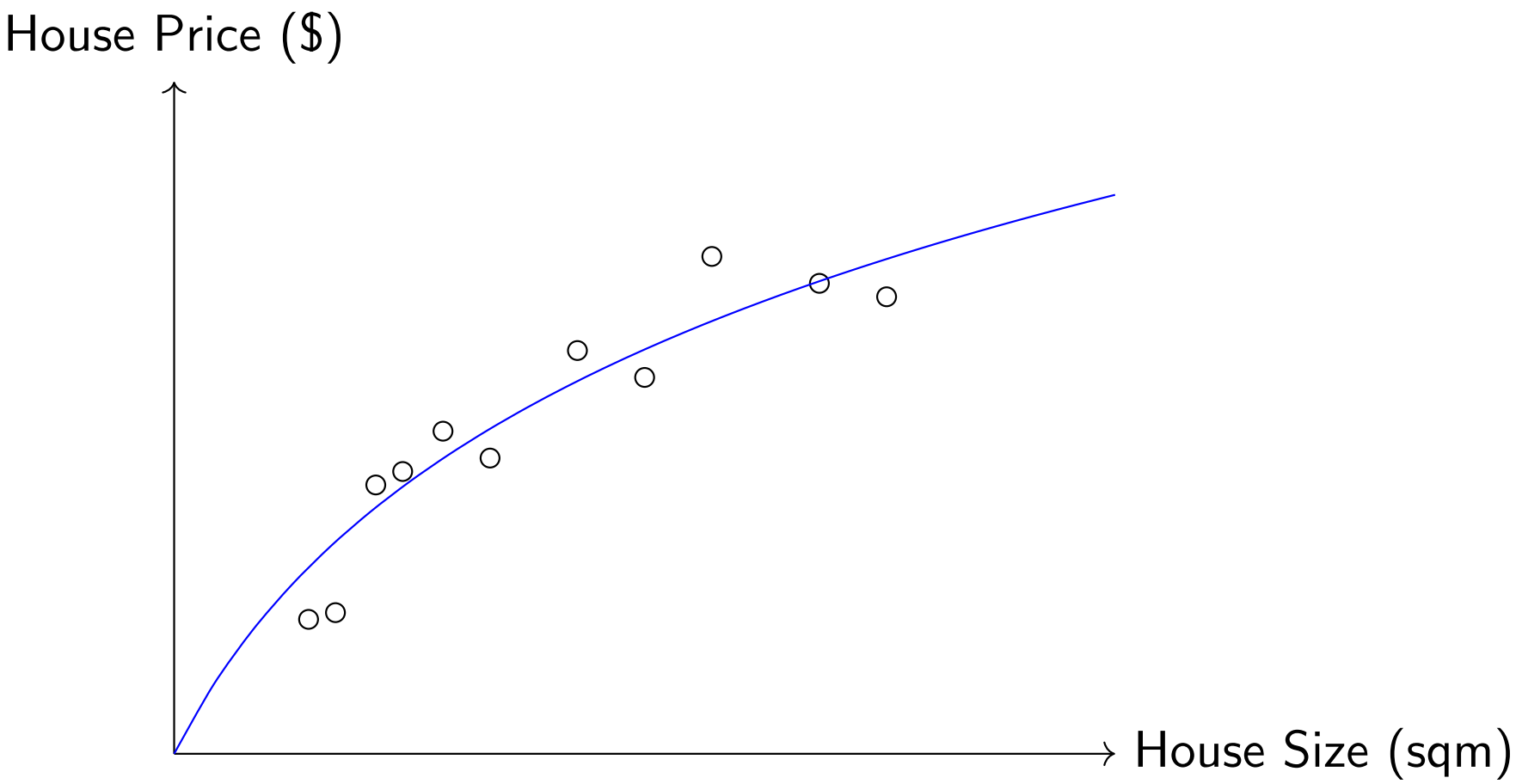
Add humans in the loop. Add guardrails and/or limit initial scope.

Summary

- ML adds complexity. Consider whether you really need it
- Make sure what you're working on is high impact, or else it might get killed

Recap of fundamental ML concepts

House Price Prediction



Machine Learning

- **Machine Learning (ML):** A subset of artificial intelligence that enables systems to learn from experience, i.e., data.
- **Training:** Models are trained using (labeled) data.
- **Testing:** Model performance is evaluated on unseen data to assess generalization capability.
- **Code + Data = Results:** In traditional software development, we code systems which process data to produce meaningful results.
- **Data + Results = Code:** In machine learning, we train models on data and results(labels) to create executable code.

Supervised Learning

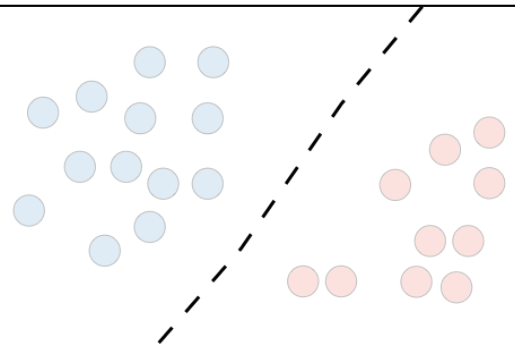
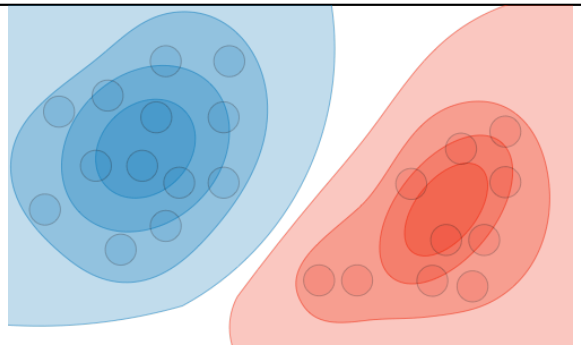
Given a set of data points $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$ with associated outcomes $\{y^{(1)}, \ldots, y^{(m)}\}$, build a model that learns how to predict y from x.

Type of Prediction

	Regression	**Classification**
Outcome	Continuous	Discrete
Examples	Linear regression	Logistic regression, SVM, Naive Bayes

Stanford cheat sheet CS229

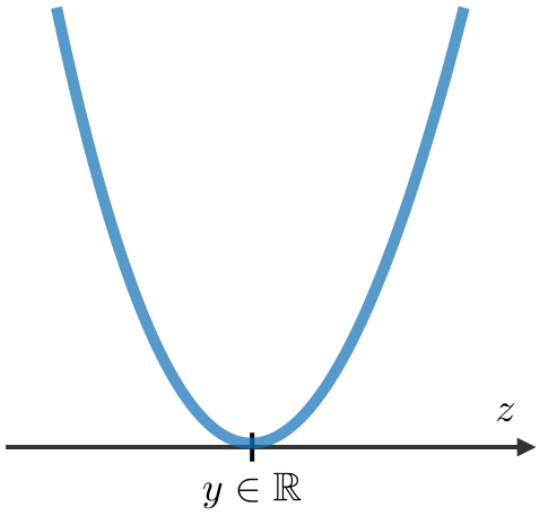
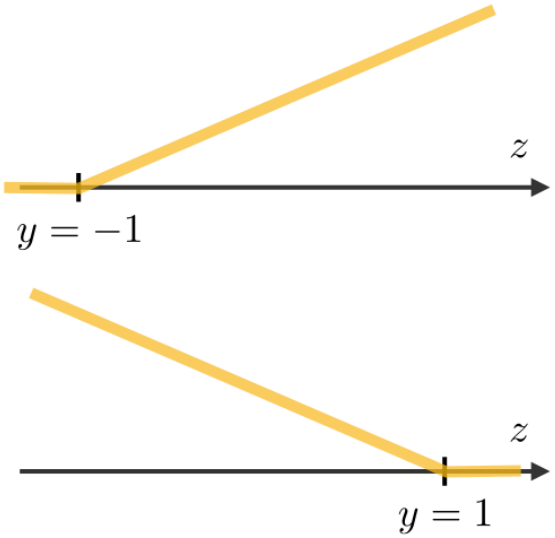
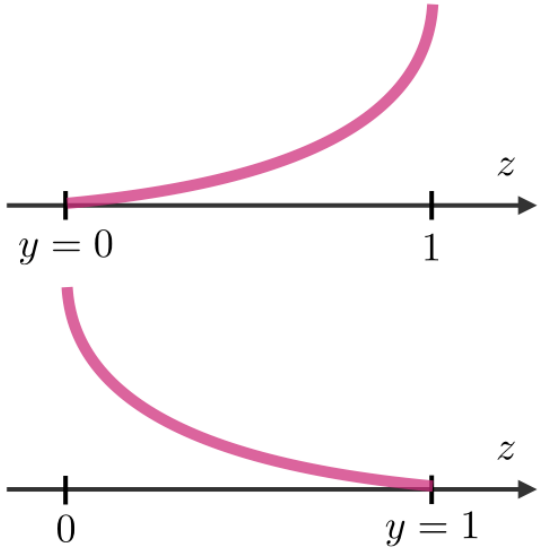
Type of Model

	Discriminative Model	**Generative Model**
Goal	Estimate $P(y\|x)$	Estimate $P(x, y)$
What's learned	Decision boundary	Probability distributions
Illustration		
Examples	Regressions, SVMs	GDA, Naive Bayes

Stanford cheat sheet CS229

Loss function

Loss function: quantify difference between predicted value z and real data value y

Least Squared Error	Hinge loss	Cross-entropy
$\frac{1}{2}(y-z)^2$	$\max(0, 1-yz)$	$-[y\log(z)+$ $(1-y)\log(1-z)]$
		

Stanford cheat sheet CS229

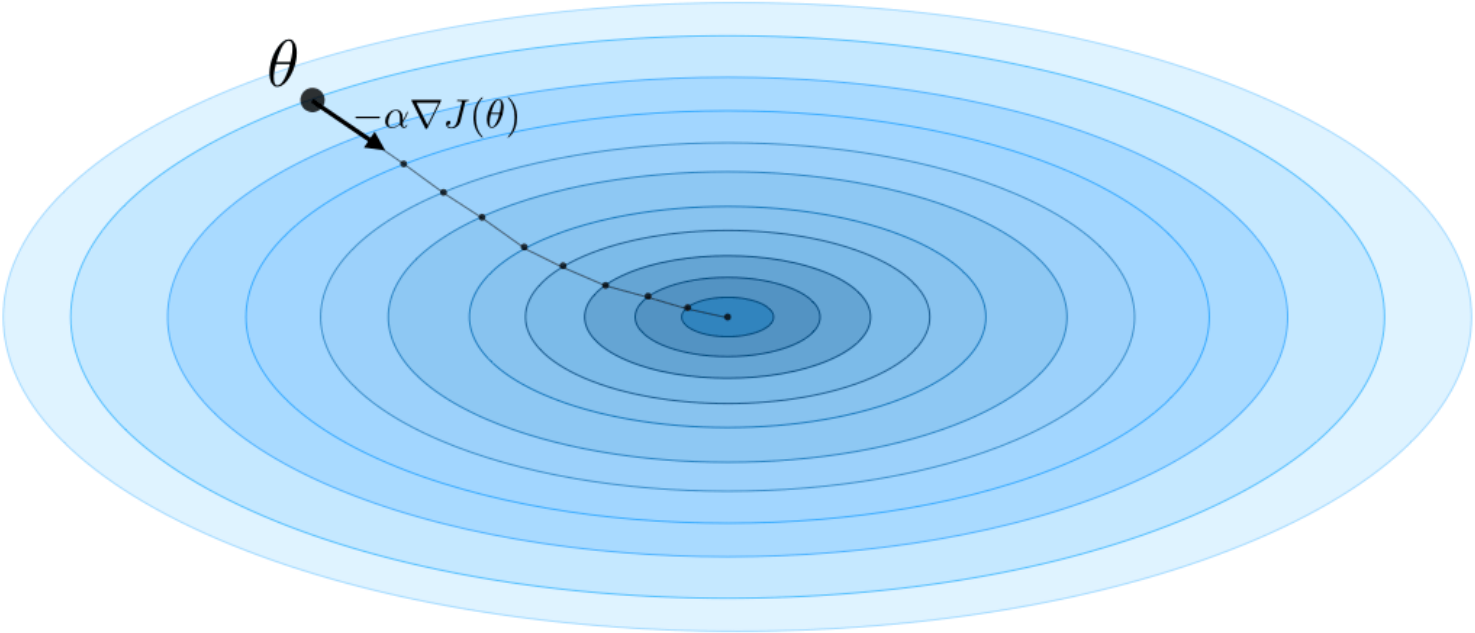
Gradient descent

Cost Function: assess the performance of a model

$$J(\theta) = \sum_{i=1}^{m} L(h_\theta(x^{(i)}), y^{(i)})$$

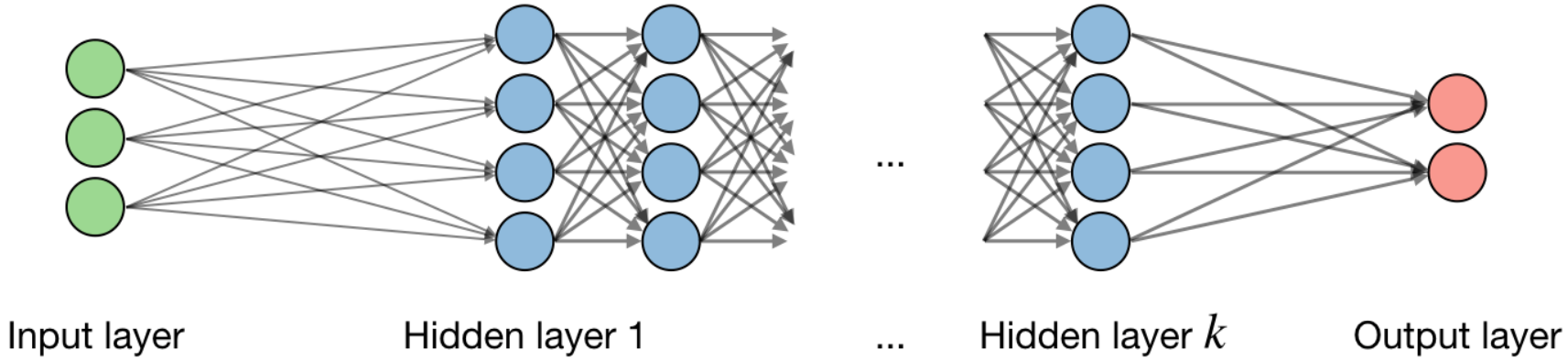
Gradient Descent: Take "one step" with learning rate α in the direction of steepest decline of the cost function.

$$\theta \leftarrow \theta - \alpha \nabla J(\theta)$$



Deep Learning

Neural networks with many (i.e. deep) number of layers.



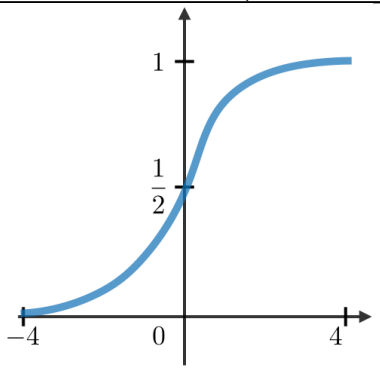
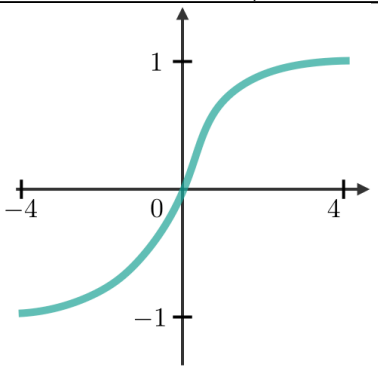
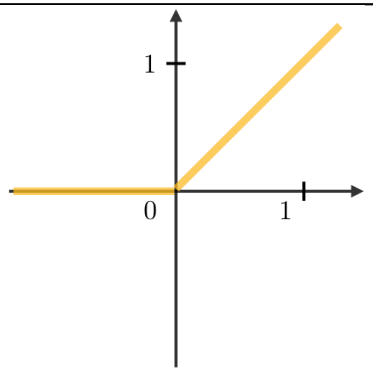
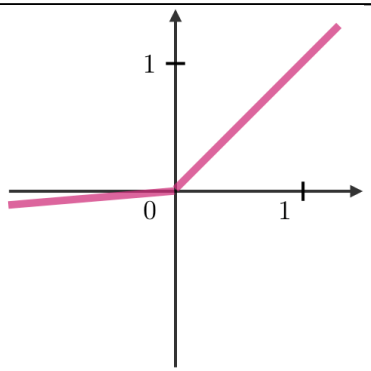
By noting i as the i-th layer of the network and j as the j-th hidden unit of the layer, we have:

$$z_j^{[i]} = w_j^{[i]T} x + b_j^{[i]}$$

where we note w, b, and z as the **weight, bias**, and **output**, respectively.

Deep learning

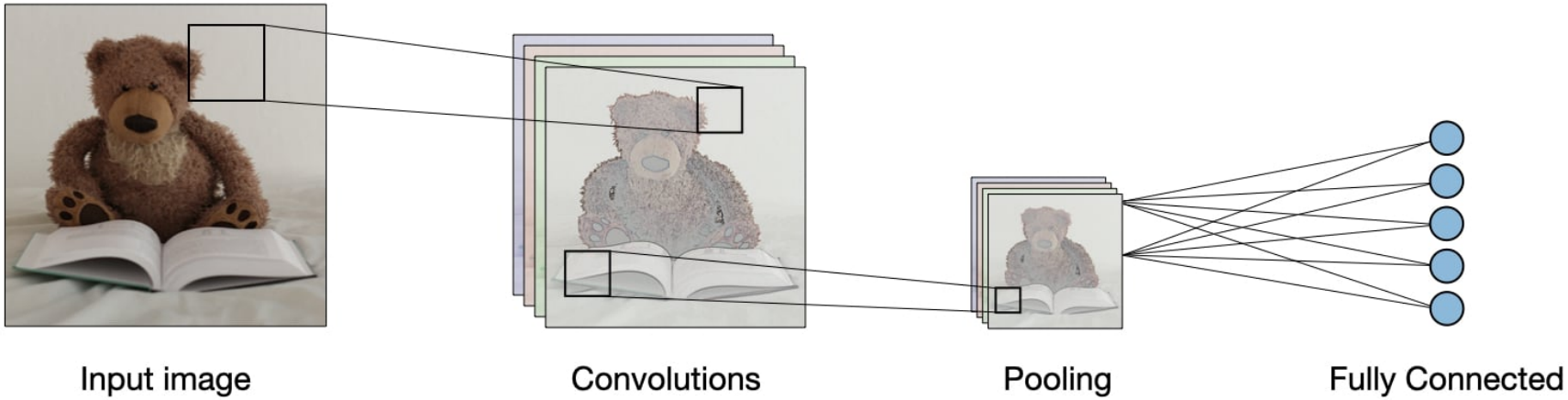
Activation function: introduce non-linearity at the output of a neuron.

Sigmoid	**Tanh**	**ReLU**	**Leaky ReLU**
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = max(0, z)$	$g(z) = max(\epsilon z, z)$
			

Stanford cheat sheet CS230

Deep learning

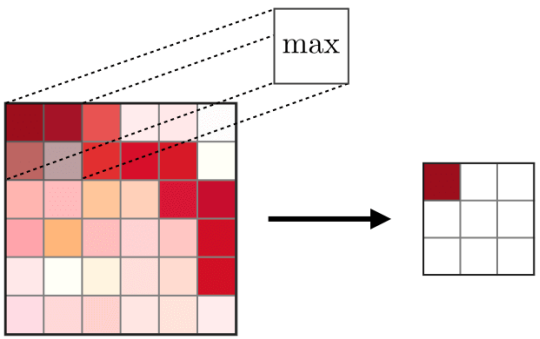
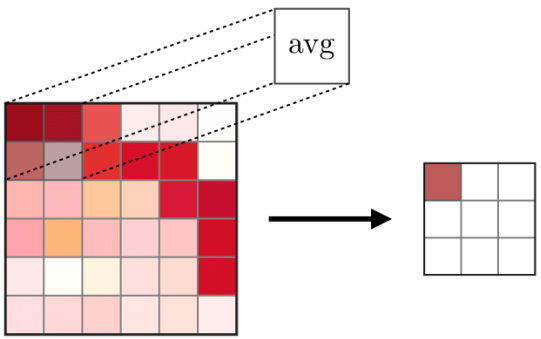
Convolutional neural networks (CNNs): specific type of neural network architecture which perform local filter operations over the input



Stanford cheat sheet CS230

Deep learning

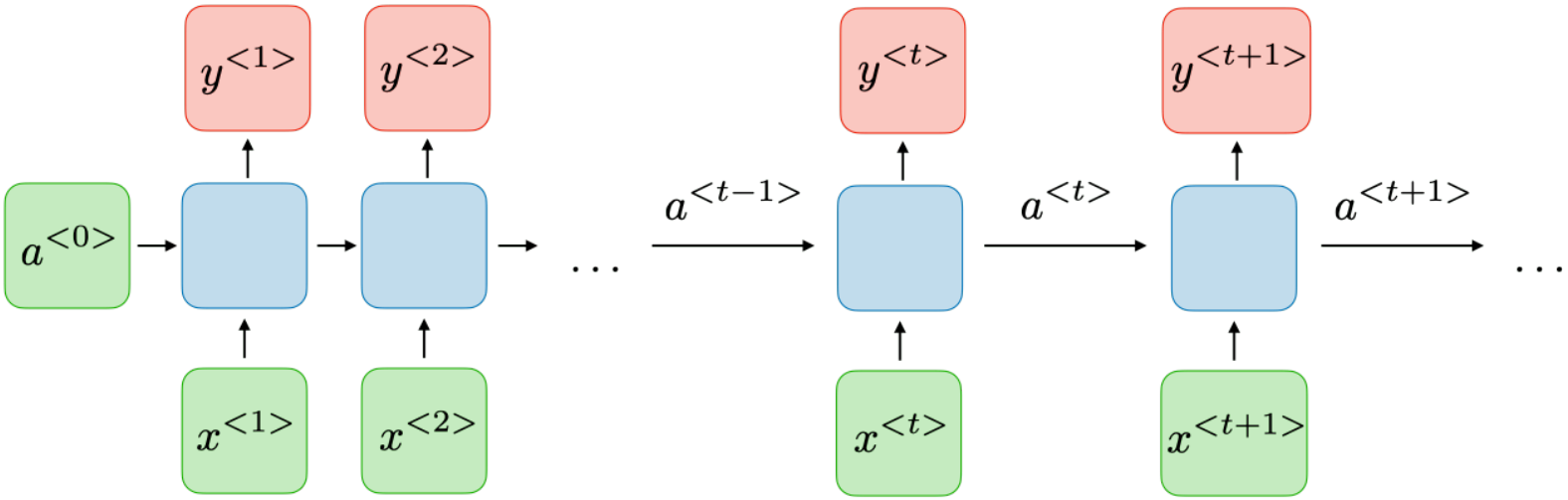
Pooling layers: perform local convolution filter operations while scanning the input

Type	Max pooling	Average pooling
Purpose	Select maximum value	Average values
Illustration		

Fully connected layers: layers with every input connected to every neuron
Stanford cheat sheet CS230

Deep learning

Recurrent neural networks (RNNs): neural network architecture which perform recursive operations over intermediate results from previous operations



Stanford cheat sheet CS230

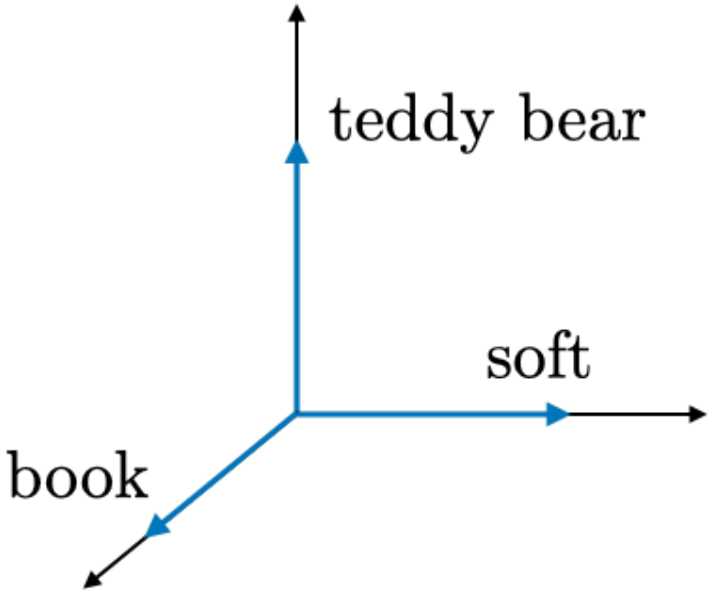
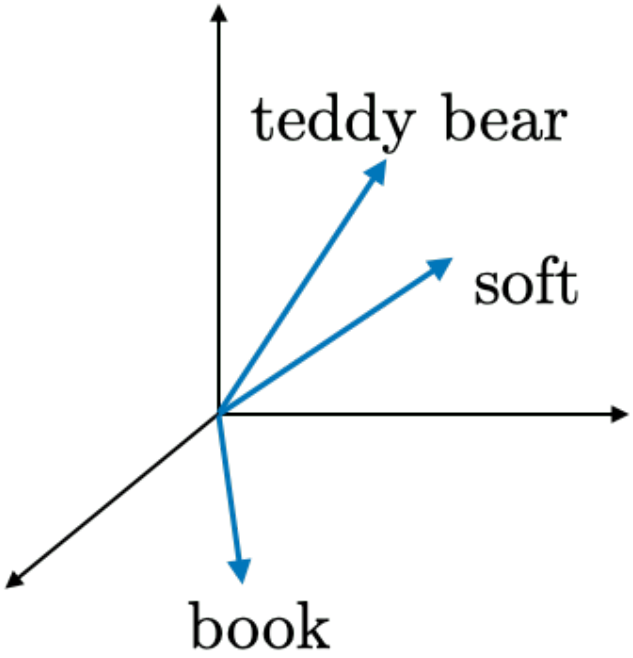
Deep learning

- Backpropagation: update neural network weights by pushing the cost signal through the layers
- Weight regularization: penalty for larger weights during training to reduce overfitting
- Dropout: technique to prevent overfitting by masking input neurons or connections at random during training
- Batchnorm: normalize the input of each mini-batch by mean and variance to reduce internal covariate shift
- Residual connections: skip connections to allow better flow of gradient
- Vanishing/exploding gradient: very small/large gradients during training, leading to numerical instabilities
- Gradient clipping: capping the maximum value for the gradient to reduce exploding gradient in practice
- Epoch: one iteration over the training data

Stanford cheat sheet CS230

Deep learning

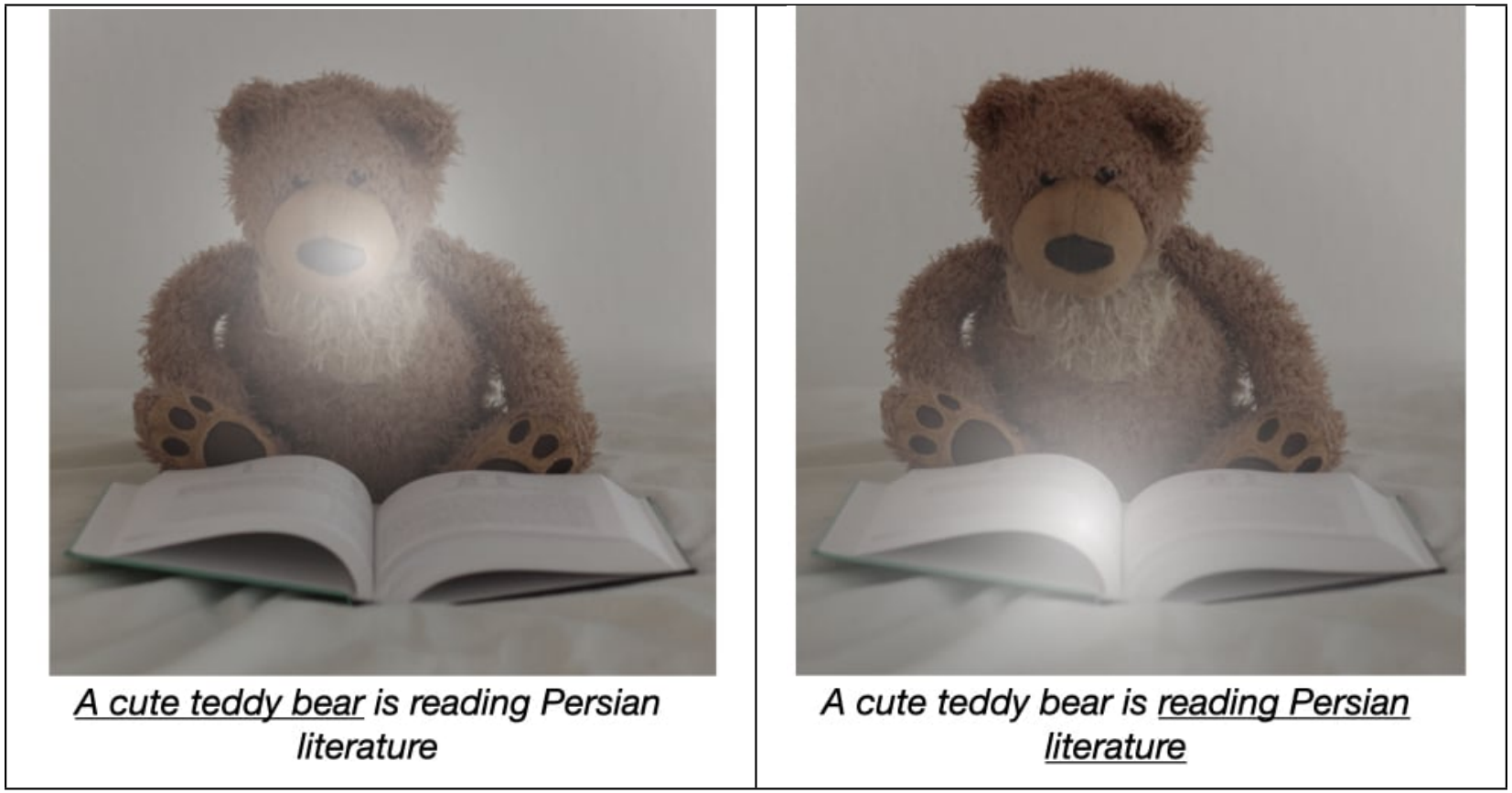
Embeddings: represent inputs as learned vectors which can capture semantic and syntactic properties

1-hot representation	Word embedding pooling
	

Stanford cheat sheet CS230

Deep learning

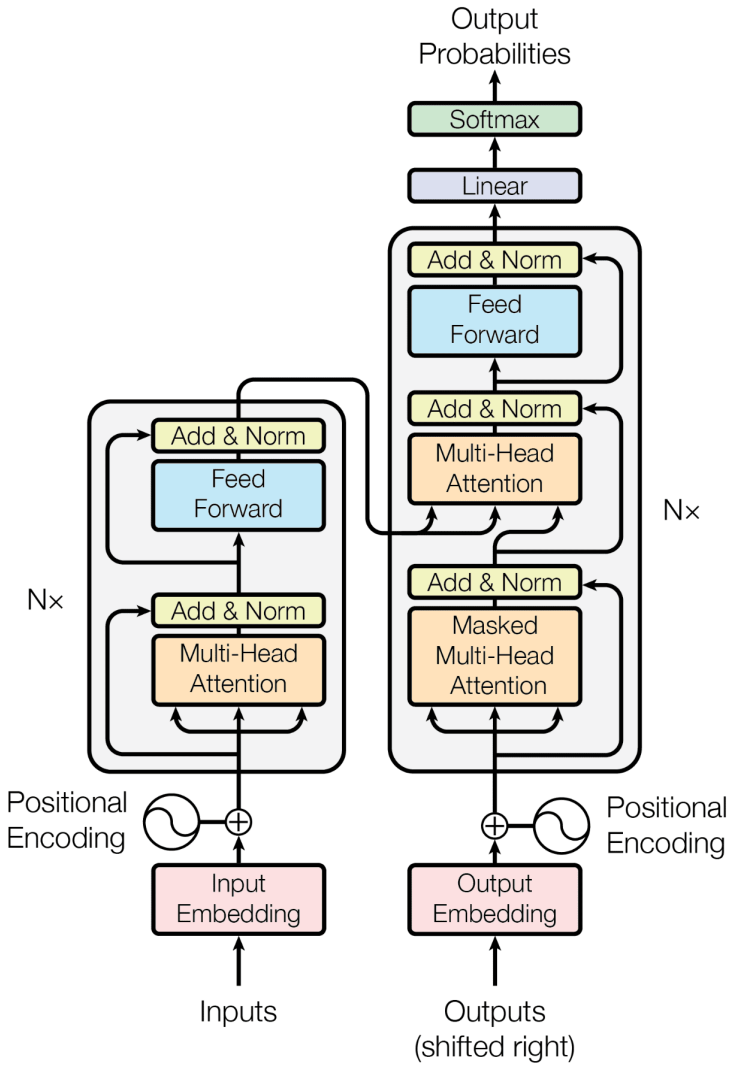
Attention: allows networks to learn a weighting of the inputs in a specific context



Stanford cheat sheet CS230

Deep learning

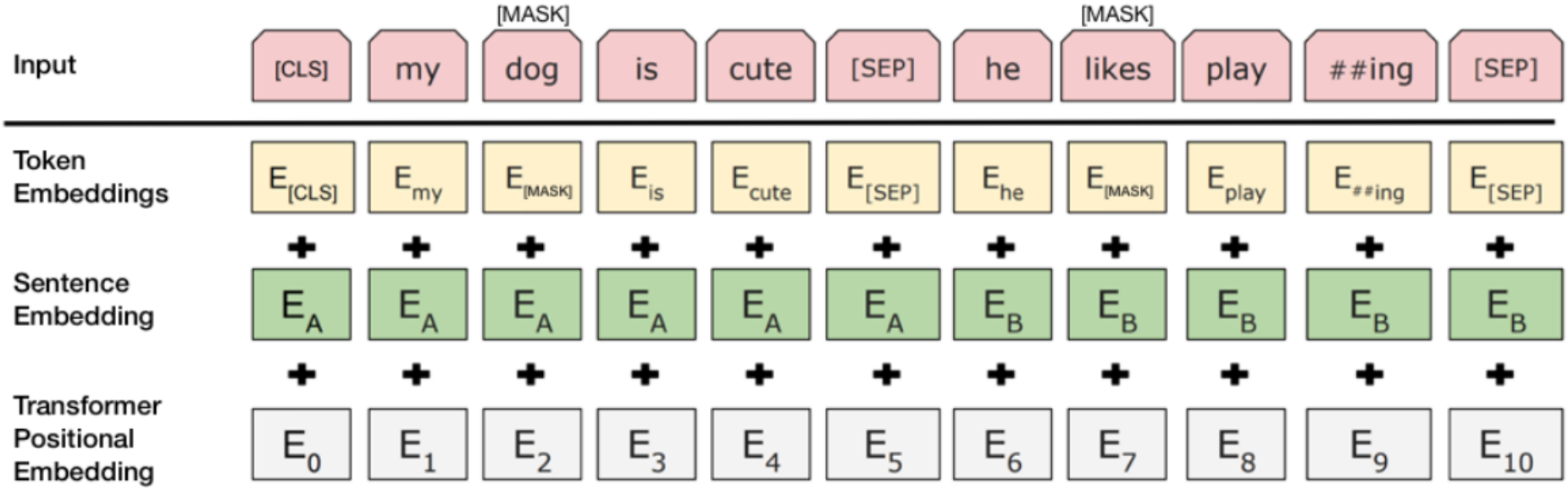
Transformers: neural network architecture which computes contextual embeddings via self-attention. Overcomes the sequential computation bottleneck and vanishing gradient problem of RNNs.



Vaswani et al. (2017)

Deep learning

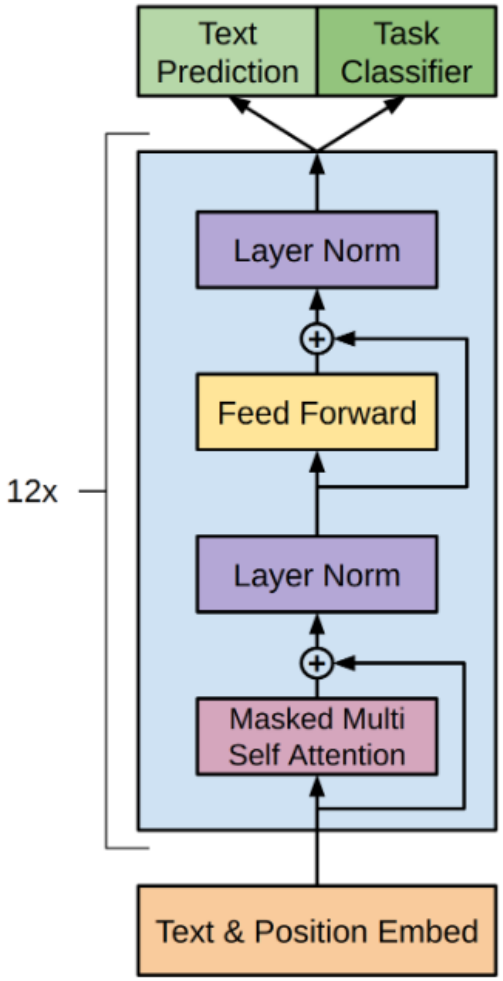
BERT: pre-trained transformer encoder model to compute context-sensitive representations.



Devlin et al. 2019

Deep learning

GPT: family of pre-trained generative transformer decoder models.



Radford et al. 2018

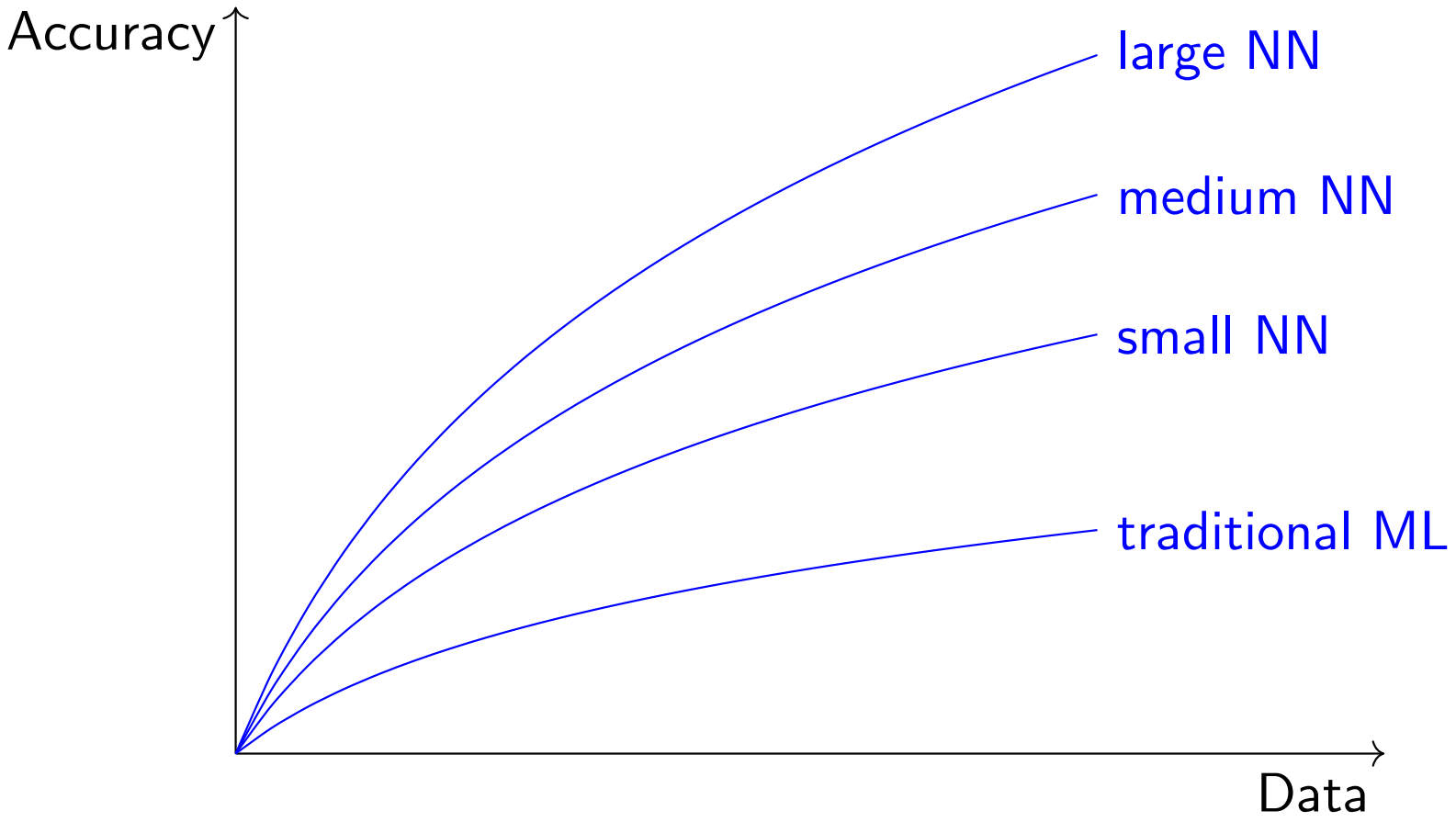
Metrics

		Predicted class	
		+	-
Actual class	+	True Positive (TP)	False Negative (FN)
	-	False Positive (FP)	True Negative (TN)

Metric	Formula
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision	$\frac{TP}{TP+FP}$
Recall (Sensitivity)	$\frac{TP}{TP+FN}$
F1 Score	$\frac{2TP}{2TP+FP+FN}$

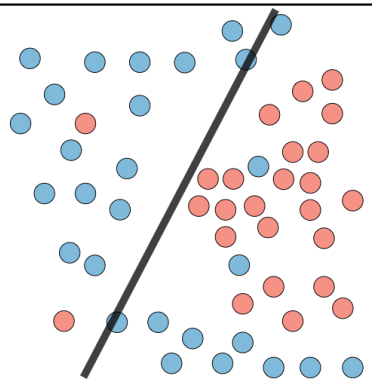
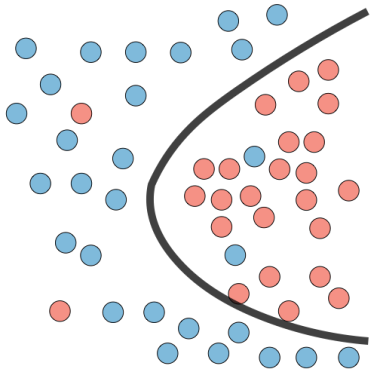
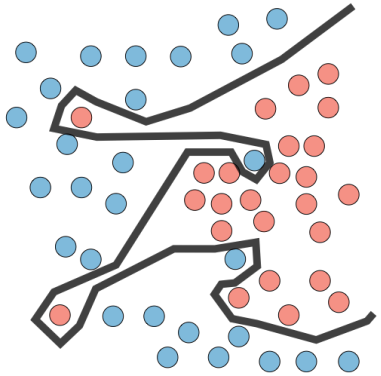
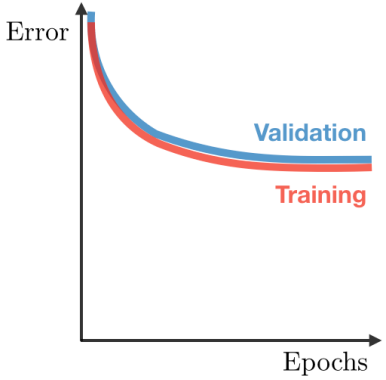
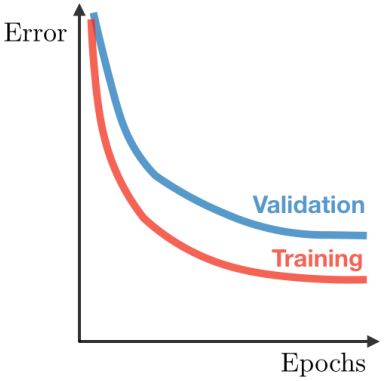
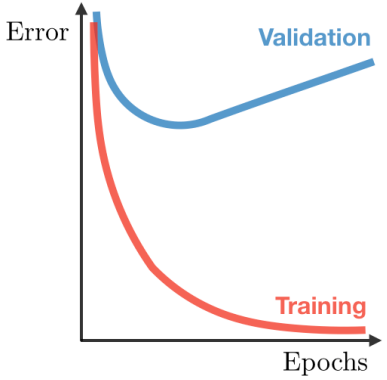
Stanford cheat sheet CS229

Deep learning models have higher capacity



deeplearning.ai

Experiments

Underfitting	Just right	Overfitting
High training error	Training error slightly lower than test error	Very low training error much lower than test error
		
		

Stanford cheat sheet CS229

Summary and Q&A

- Key takeaways
  - Tremendous progress in ML and increased adoption in industry
  - ML increases complexity: make sure you really need it and the value is high enough
  - Design your application to cope with imperfect ML, e.g., via humans in the loop, and implement data flywheel to improve ML over time
- Open discussion and questions
  - What do you consider successful ML applications today?
  - What ML systems are you using in your life?
  - What ML systems would you like to see being built?

Readings and additional literature

- **Textbooks:**
  - Chip Huyen. *Designing Machine Learning Systems*. O'Reilly, 2022.
  - Antje Barth, Chris Fregly. Data Science on AWS: Implementing End-to-End, Continuous AI and Machine Learning Pipelines. O'Reilly, 2021.
  - Chris Fregly, Antje Barth, et al. Generative AI on Aws: Building Context-Aware Multimodal Reasoning Applications. O'Reilly, 2023.
  - Kenji Lefevre. *Introducing ML Ops*. O'Reilly, 2020.
  - Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- **Academic Papers:**
  - D. Sculley et al. *Machine Learning: The High-Interest Credit Card of Technical Debt*.
- **Case studies:**
  - Case study collection from Designing Machine Learning Systems

Course Logistics

- Schedule and Assignments
  - TBC
- Office hours
  - Send email to christopher_teo@alumni.sutd.edu.sg